Epic Games
Game Designer Internship Submission



Chain Lightning Ability that feels powerful and satisfying (made multiplayer-ready)

Jorge Paladines

Contents

The Ability Setup 3

Attribute Explanation 4

Aiming 6

The bIsAiming replicated variable 8

Firing the Projectile 10

Not currently aiming 13

Actively Aiming 14

The Projectile 15

The Chain Lightning Ability 18

Settings 18

The Algorithm 18

Visual and Sound Effects 30

GC_FireLightning 30

GC_LightningLink 31

GC_ElectrocutedEffect 36

The Explosion 38

Animation 39

Upper and Lower Body 39

Bone Modification 40

User Interface 43

Crosshair 43

Cooldown Timer 44

The Ability Setup

I first created the Attributes for the Ability itself, in C++:

```
/** Theorical damage for the enemy */
UPROPERTY(BlueprintReadOnly, Category = "Attributes", ReplicatedUsing=OnRep_Damage)
FGameplayAttributeData Damage;
ATTRIBUTE_ACCESSORS_BASIC(UChainLightningAttributeSet, Damage);

/** Radius of the hit detection */
UPROPERTY(BlueprintReadOnly, Category = "Attributes", ReplicatedUsing=OnRep_BounceRadius)
FGameplayAttributeData BounceRadius;
ATTRIBUTE_ACCESSORS_BASIC(UChainLightningAttributeSet, BounceRadius);

/** Max amount of bounces that the chain can perform */
UPROPERTY(BlueprintReadOnly, Category = "Attributes", ReplicatedUsing=OnRep_MaxBounces)
FGameplayAttributeData MaxBounces;
ATTRIBUTE_ACCESSORS_BASIC(UChainLightningAttributeSet, MaxBounces);
```

[attribute header's variables – not all attributes]

And I made sure all of them were replicated with OnRep:

```
UFUNCTION()
void OnRep_Damage(const FGameplayAttributeData& OldValue) const
{
    GAMEPLAYATTRIBUTE_REPNOTIFY(UChainLightningAttributeSet, Damage, OldValue);
}

UFUNCTION()
void OnRep_BounceRadius(const FGameplayAttributeData& OldValue) const
{
    GAMEPLAYATTRIBUTE_REPNOTIFY(UChainLightningAttributeSet, BounceRadius, OldValue);
}
```

[OnRep functions for the variables]

Here's all the DOREPLIFETIMEs for each variable:

```
void UChainLightningAttributeSet::GetLifetimeReplicatedProps(TArray<class FLifetimeProperty>& OutLifetimeProps) const
{
    Super::GetLifetimeReplicatedProps( [&] OutLifetimeProps);
    DOREPLIFETIME_CONDITION_NOTIFY(UChainLightningAttributeSet, Damage, COND_None, REPNOTIFY_Always);
    DOREPLIFETIME_CONDITION_NOTIFY(UChainLightningAttributeSet, MaxBounces, COND_None, REPNOTIFY_Always);
    DOREPLIFETIME_CONDITION_NOTIFY(UChainLightningAttributeSet, BounceRadius, COND_None, REPNOTIFY_Always);
    DOREPLIFETIME_CONDITION_NOTIFY(UChainLightningAttributeSet, BounceDelay, COND_None, REPNOTIFY_Always);
    DOREPLIFETIME_CONDITION_NOTIFY(UChainLightningAttributeSet, ChainLinkDelay, COND_None, REPNOTIFY_Always);
    DOREPLIFETIME_CONDITION_NOTIFY(UChainLightningAttributeSet, EffectDuration, COND_None, REPNOTIFY_Always);
    DOREPLIFETIME_CONDITION_NOTIFY(UChainLightningAttributeSet, StunDuration, COND_None, REPNOTIFY_Always);
    DOREPLIFETIME_CONDITION_NOTIFY(UChainLightningAttributeSet, ChainAmount, COND_None, REPNOTIFY_Always);
    DOREPLIFETIME_CONDITION_NOTIFY(UChainLightningAttributeSet, Cooldown, COND_None, REPNOTIFY_Always);
}
```

All the attributes are to be modified inside a Data Table. This will make playtest iteration much easier:

	Row Name	Base Value
1	ChainLightningAttributeSet.Damage	50.000000
2	ChainLightningAttributeSet.MaxBounces	30.000000
3	ChainLightningAttributeSet.BounceRadiu	1000.000000
4	ChainLightningAttributeSet.BounceDelay	0.050000
5	ChainLightningAttributeSet.ChainLinkDe	0.020000
6	ChainLightningAttributeSet.StunDuratio	3.000000
7	ChainLightningAttributeSet.EffectDurati	2.000000
8	ChainLightningAttributeSet.ChainAmour	5.000000
9	ChainLightningAttributeSet.Cooldown	5.000000

Attribute Explanation

These are all the values and explanations:

- Damage
  - Theorical damage for the enemy. Not used for this implementation
- MaxBounces
  - The max amount of total allowed bounces to any other enemy
- BounceRadius
  - The radius that each enemy will try to look for another target in a sphere
- BounceDelay
  - A delay added for the lightning for each progression level frontier. A Frontier is all the enemies to be strike by the lightning on a corresponding foreach loop.
- ChainLinkDelay
  - A delay added for each lightning chain connection. To add an extra behavior to it
- EffectDuration
  - The time the lightning chain effect lasts on the screen
- StunDuration
  - The time that the enemy will have the electrocution visual effects on it, which is a theorical stun
- ChainAmount
  - The amount of chain connections allowed from one enemy to the next level
- Cooldown
  - The cooldown duration of the ability

I added the Ability System Component and the ChainLightningAttributeSet to the player character's header:

```
UPROPERTY(VisibleAnywhere, BlueprintReadOnly, Category="AbilitySystem")
UAbilitySystemComponent* AbilitySystemComponent;

UPROPERTY(Transient, BlueprintReadOnly, Category = "AbilitySystem")
class UChainLightningAttributeSet* ChainLightningAttributeSet;
```

```
UFUNCTION(BlueprintCallable, Category = "AbilitySystem")
virtual UAbilitySystemComponent* GetAbilitySystemComponent() const override;

UFUNCTION(BlueprintCallable, Category = "AbilitySystem")
void CancelAbilitiesWithTags(const FGameplayTagContainer& WithTags);  1 bluepri
```

As well as the Fire and Aim input actions:

```
/** Fire Chain Lightning Input Action */
UPROPERTY(EditAnywhere, Category="Input")
UInputAction* FireAction;  "IA_Fire" (Default__BP

UPROPERTY(EditAnywhere, Category="Input")
UInputAction* AimAction;  "IA_Aim" (Default__BP_
void Fire(const FInputActionValue& Value);

void AimStart(const FInputActionValue& Value);

void AimEnd(const FInputActionValue& Value);
```

```
// Fire
EnhancedInputComponent->BindAction(FireAction, ETriggerEvent::Started,  Object: this, &AChainLightningPowerCharacter::Fire);

// Aim
EnhancedInputComponent->BindAction(AimAction, ETriggerEvent::Started,  Object: this, &AChainLightningPowerCharacter::AimStart);
EnhancedInputComponent->BindAction(AimAction, ETriggerEvent::Completed,  Object: this, &AChainLightningPowerCharacter::AimEnd);
```

The Fire Action is BlueprintImplementableEvent because the Chain Lightning Ability needed to be implemented in Blueprints, while the Aim Start and End are implemented in C++:

```
UFUNCTION(BlueprintImplementableEvent, BlueprintCallable, Category="Input")
void DoFire();  1 blueprint usage  Implemented in 1 blueprint

UFUNCTION(BlueprintCallable, Category="Input")
void DoAimStart();  No blueprint usages

UFUNCTION(BlueprintCallable, Category="Input")
void DoAimEnd();  No blueprint usages
```

Aiming

Left button hold and release call the functions accordingly:

```
void AChainLightningPowerCharacter::AimStart(const FInputActionValue& Value)
{
    DoAimStart();
}

void AChainLightningPowerCharacter::AimEnd(const FInputActionValue& Value)
{
    DoAimEnd();
}
```

Which are also BlueprintCallable.
And they call the Ability itself:

```
void AChainLightningPowerCharacter::DoAimStart()
{
    if (!AbilitySystemComponent)
    {
        return;
    }

    FGameplayTagContainer Tag;
    Tag.AddTag(FGameplayTag::RequestGameplayTag(TEXT("Abilities.Power.Aim")));
    AbilitySystemComponent->TryActivateAbilitiesByTag(Tag);
}
void AChainLightningPowerCharacter::DoAimEnd()
{
    if (!AbilitySystemComponent)
    {
        return;
    }

    FGameplayTagContainer Tag;
    Tag.AddTag(FGameplayTag::RequestGameplayTag(TEXT("Abilities.Power.Aim")));
    AbilitySystemComponent->CancelAbilities( WithTags: &Tag);
}
```

GA_Aim.h has AimAbilityTag which will hold the Tag associated to call the ability, and AimAbilityStatusTag for the Tag assigned to the player while the ability is active.

```
UGA_Aim::UGA_Aim()
{
    InstancingPolicy = EGameplayAbilityInstancingPolicy::InstancedPerActor;

    AimAbilityTag = FGameplayTag::RequestGameplayTag(FName("Abilities.Power.Aim"));
    AimAbilityStatusTag = FGameplayTag::RequestGameplayTag(FName("Abilities.Status.IsAiming"));

    AbilityTags.AddTag(AimAbilityTag);
    ActivationOwnedTags.AddTag(AimAbilityStatusTag);

    // Prevent this ability from being activated again while it's already active
    BlockAbilitiesWithTag.AddTag(AimAbilityTag);
}
```

All abilities are Instanced Per Actor for them to be played on each player's instance of the game.

The Aim ability calls a SetAiming function in the character, and sets the controller rotation accordingly for the character to move with the camera aim.
It also adds the Crosshair to the Viewport via the Player Controller.

```
if (AChainLightningPowerCharacter* Character = Cast<AChai
{
    Character->SetAiming( bAim: true);
    Character->bUseControllerRotationYaw = true;
}

if (AChainLightningPowerPlayerController* PC = Cast<AChai
{
    PC->ShowAimingUI();
}
```

```
void AChainLightningPowerPlayerController::ShowAimingUI()
{
    if (!IsLocalPlayerController() || !AimingUIClass)
    {
        return;
    }

    if (!AimingUI)
    {
        AimingUI = CreateWidget<UAimingUI>( OwningObject: this, AimingUIClass);
    }

    if (AimingUI && !AimingUI->IsInViewport())
    {
        AimingUI->AddToViewport();
    }
}
```

The bIsAiming replicated variable

To allow all players to see all characters aiming, the **bIsAiming** variable uses OnRep Notify:

```
UPROPERTY(ReplicatedUsing = OnRep_IsAiming)
bool bIsAiming;
```

With its corresponding functions:
Protected:

```
UFUNCTION()
void OnRep_IsAiming(const bool OldValue);

UFUNCTION(Server, Reliable)
void ServerSetIsAiming(bool bAim);
```

Public:

```
UFUNCTION(BlueprintCallable, Category = "Input")
void SetAiming(bool bAim);  ⓤNo blueprint usages
```

They work like this:

1. The **bIsAiming** function uses **COND_SkipOwner** since the player is allowed to dictate if they are aiming or not, and don't need server confirmation/validation.

```
DOREPLIFETIME_CONDITION_NOTIFY(AChainLightningPowerCharacter, bIsAiming, COND_SkipOwner, REPNOTIFY_OnChanged);
```

2. Implement the server function, only meant to be called with Authority, and not public

```
void AChainLightningPowerCharacter::ServerSetIsAiming_Implementation(bool bAim)
{
    SetAiming(bAim);
}
```

This calls the SetAiming public function.

3. The **SetAiming** function just sets the new value of the Aiming state, and if the player that called it doesn't have authority, tells the server to broadcast the value to it and other players.

```
void AChainLightningPowerCharacter::SetAiming(bool bAim)
{
    bIsAiming = bAim;
    if (!HasAuthority())
    {
        ServerSetIsAiming(bAim);
    }
}
```

This ensures a client and server communication where the client doesn't wait for the server to set its aiming state, while all players, no matter who called the function, are able to see each other aiming.

And since SetAiming is a public BlueprintCallable, this can be used anywhere with confidence.

4. **OnRep_IsAiming** has no extra implementation

```
void AChainLightningPowerCharacter::OnRep_IsAiming(const bool OldValue)
{
    //
}
```

The character also has code inside Tick to interpolate the camera positions from aiming and not aiming, so it moves between the positions instead of jumping from one to the other (with modifiable settings such as each state position and interpolation speed)

```
void AChainLightningPowerCharacter::Tick(float DeltaSeconds)
{
    Super::Tick(DeltaSeconds);

    // The camera only matters to the player who owns it
    if (!IsLocallyControlled() || !CameraBoom)
    {
        return;
    }

    const float TargetArmLength = bIsAiming ? AimArmLength : DefaultArmLength;
    const FVector TargetSocketOffset = bIsAiming ? AimSocketOffset : DefaultSocketOffset;

    if (AimInterpSpeed <= 0.0f)
    {
        CameraBoom->TargetArmLength = TargetArmLength;
        CameraBoom->SocketOffset = TargetSocketOffset;
        return;
    }

    CameraBoom->TargetArmLength = FMath::FInterpTo( Current: CameraBoom->TargetArmLength, TargetArmLength, DeltaTime: DeltaSeconds, AimInterpSpeed);
    CameraBoom->SocketOffset = FMath::VInterpTo( Current: CameraBoom->SocketOffset, TargetSocketOffset, DeltaTime: DeltaSeconds, AimInterpSpeed);
}
```

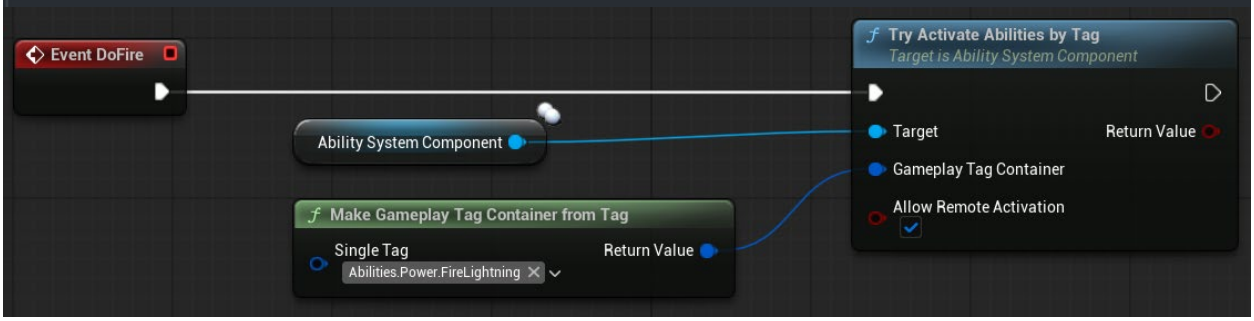
Firing the Projectile

I first added a **GA_FireChainLightning** ability made in Blueprints. This ability calls the projectile actor that will make contact with an enemy to trigger the chain lightning.

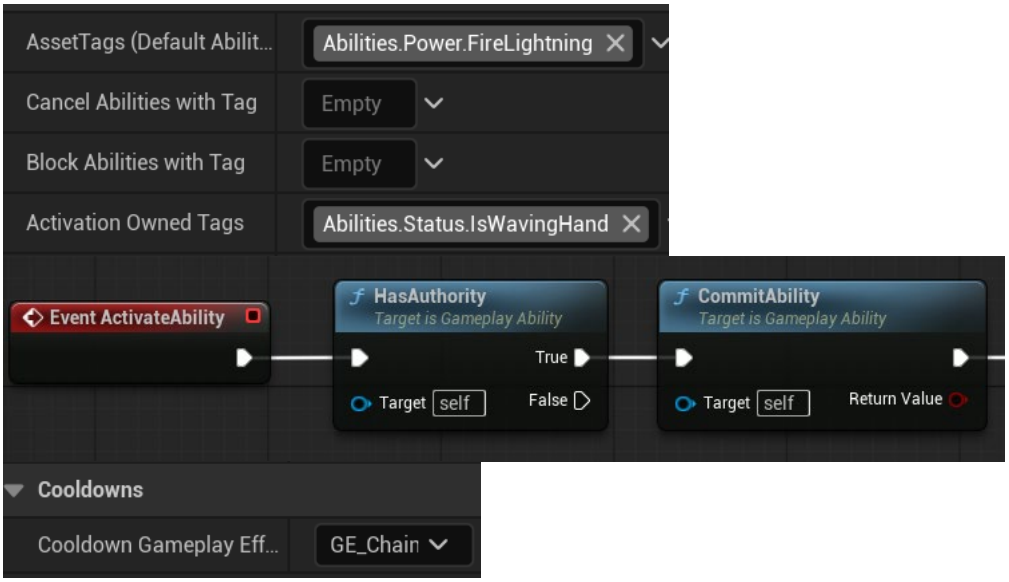
The EnhancedInputComponent calls the protected function that calls the BlueprintCallable function that triggers the Gameplay Ability:

```
// Fire
EnhancedInputComponent->BindAction(FireAction, ETriggerEvent::Started, Object: this, &AChainLightningPowerCharacter::Fir
```

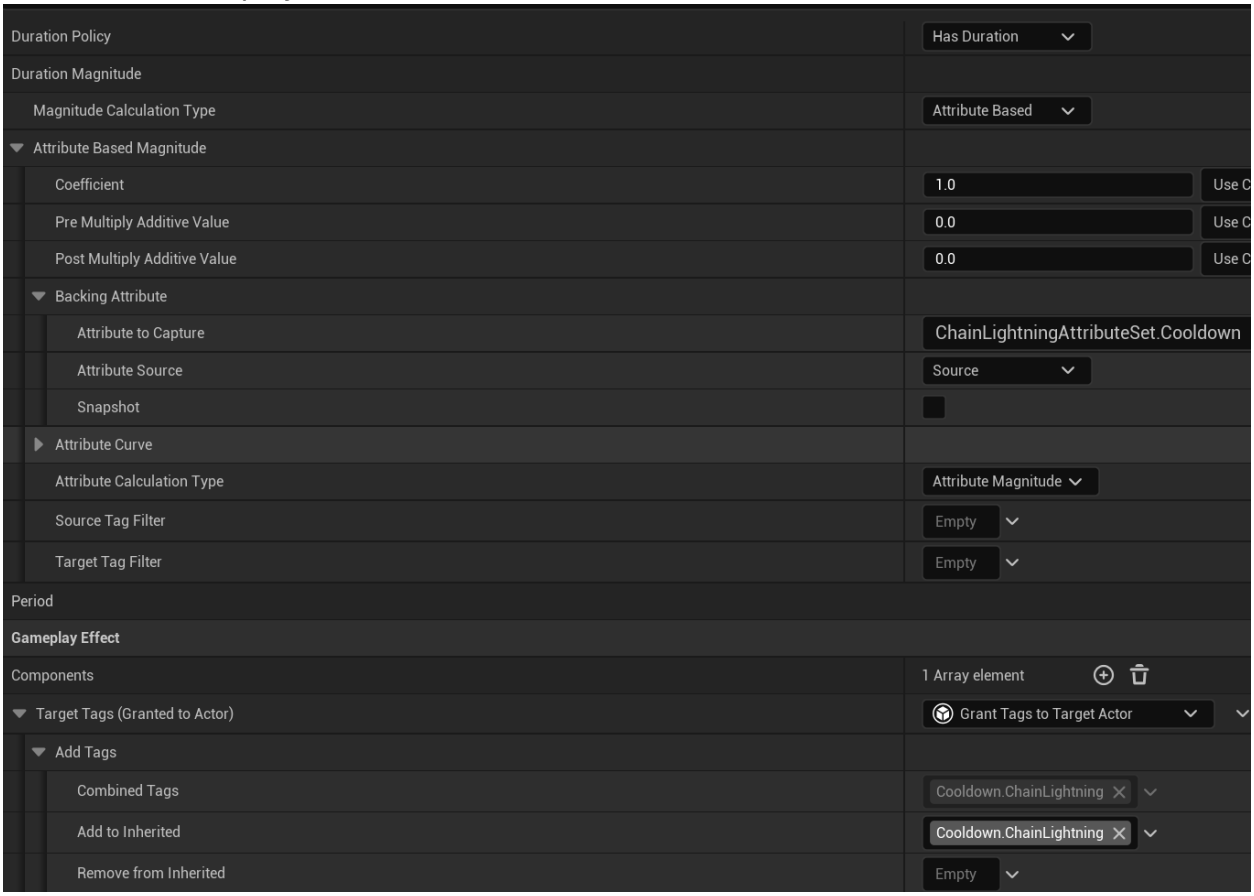
```
void AChainLightningPowerCharacter::Fire(const FInputActionValue& Value)
{
    DoFire();
}
```



The Ability First confirms that it has authority, and calls CommitAbility to be able to have cooldown.

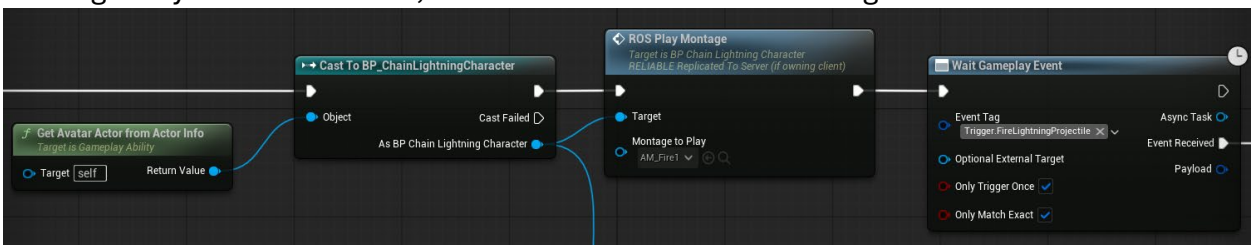


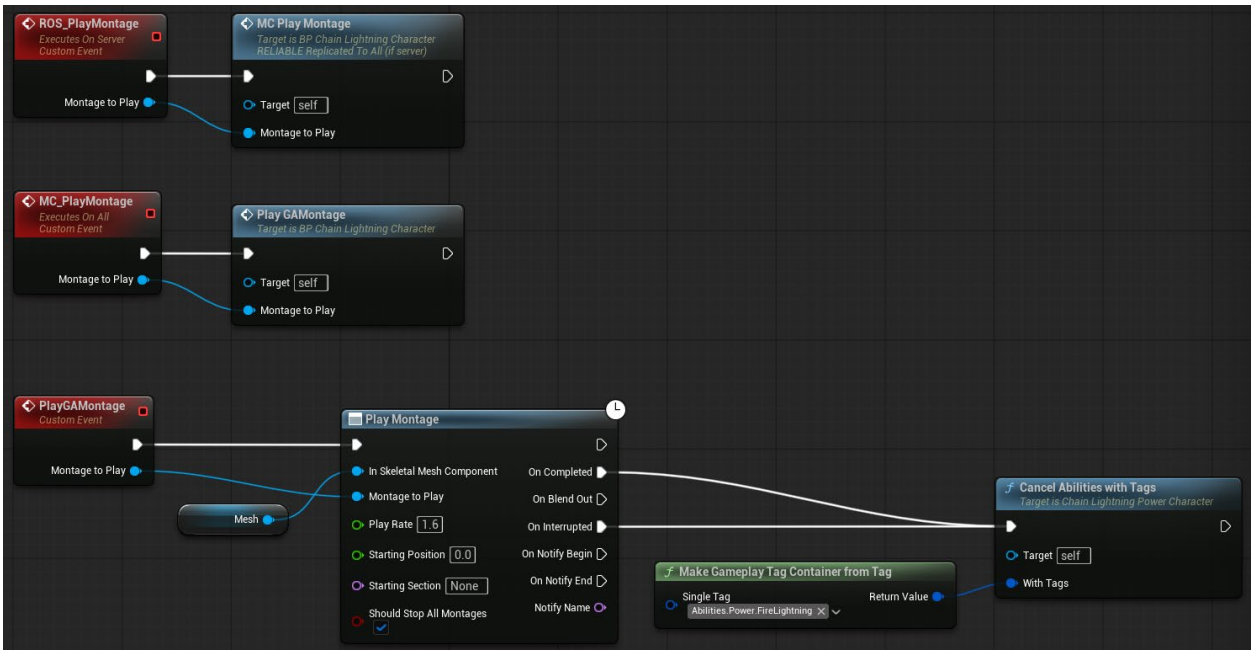
Here is the Gameplay Effect for the Cooldown:



Which uses the Attribute on the Data Table.

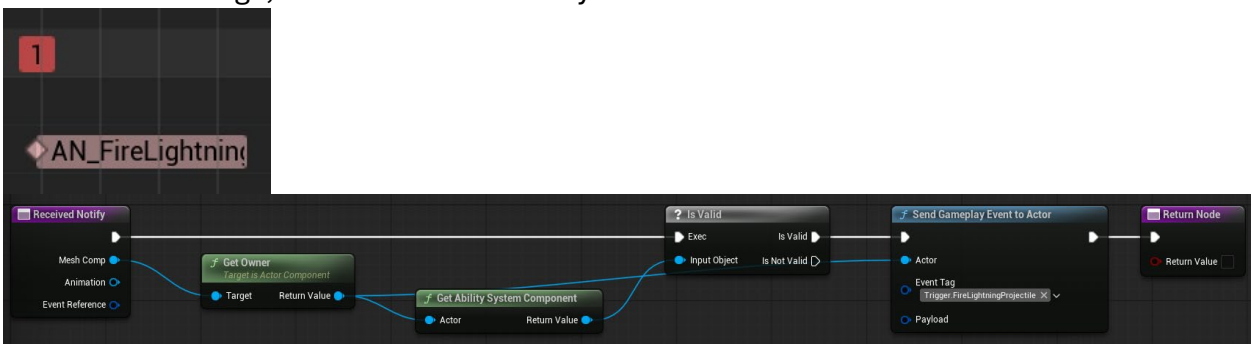
The **GA_FireChainLightning** works by calling a Run On Server function that just calls a Montage Play on the character, and then it waits for an Event Tag:





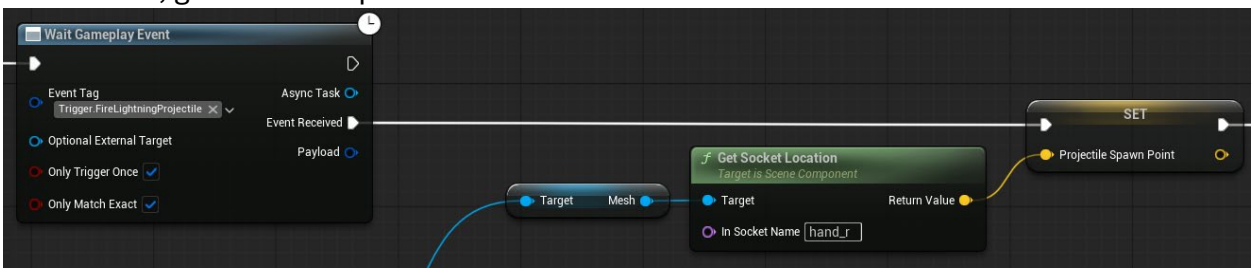
Essentially, just fire the montage, making sure any current connected player sees the event, and when the montage completes or gets interrupted, cancel the ability via my own implemented function.

Inside the montage, there is an Anim Notify:

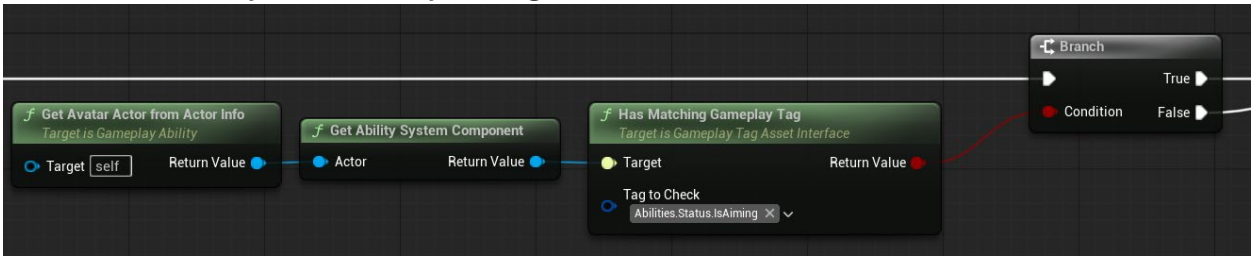


It just sends the Gameplay Event Actor that the **GA_FireChainLightning** is waiting for. This makes it so the projectile is spawned on a specific frame of the animation.

After which, get the world position of the hand socket:



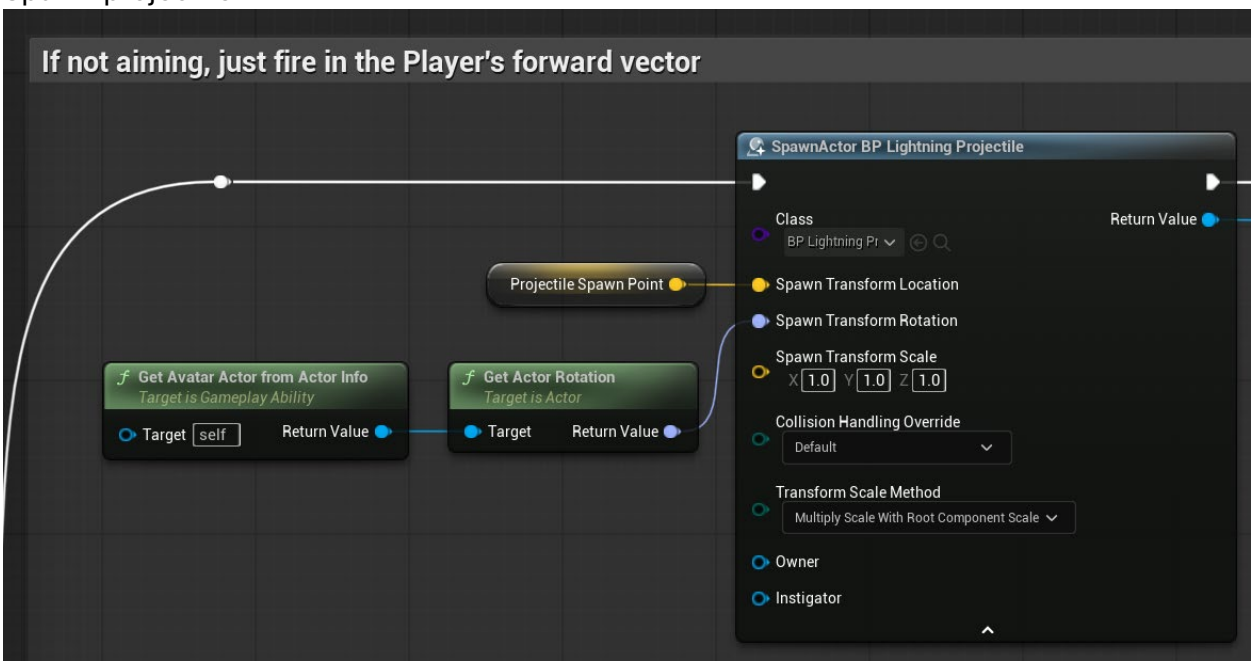
And validate if they are currently aiming or not:



Not currently aiming

Fire the projectile in the forward's vector of the player, independent on where they are looking.

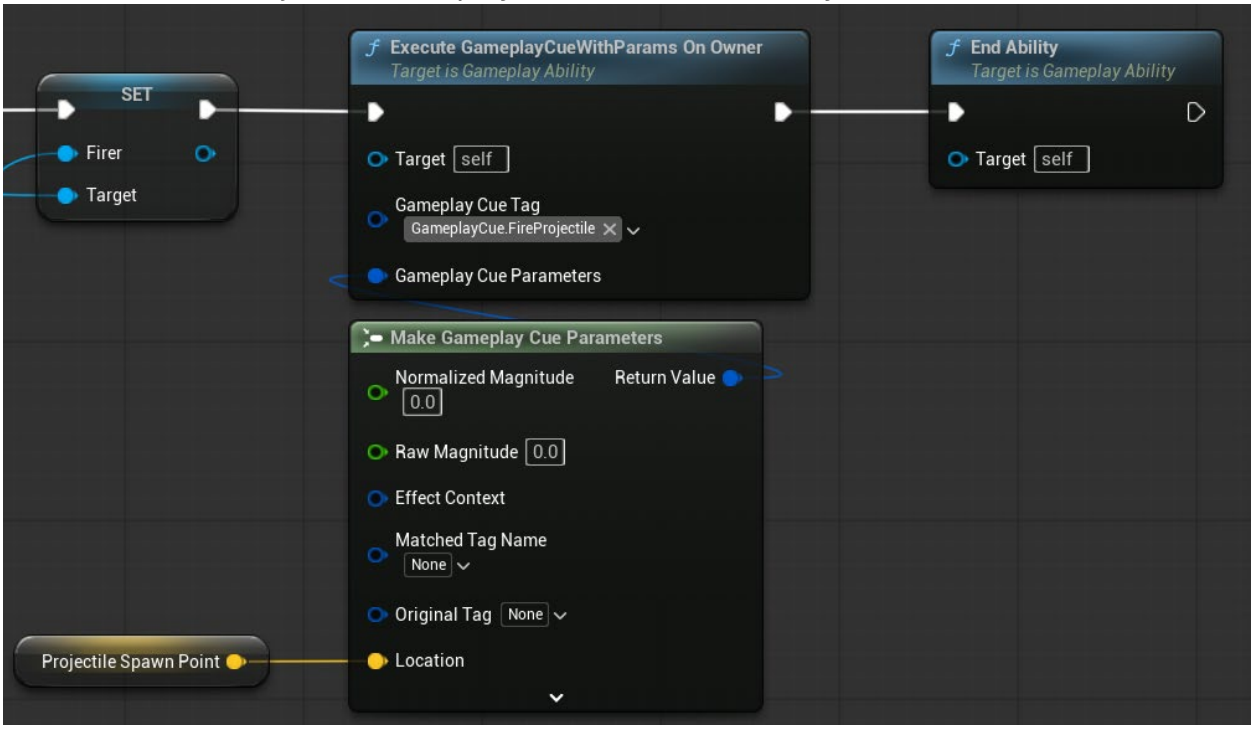
Spawn projectile:



Tell the projectile who fired it:

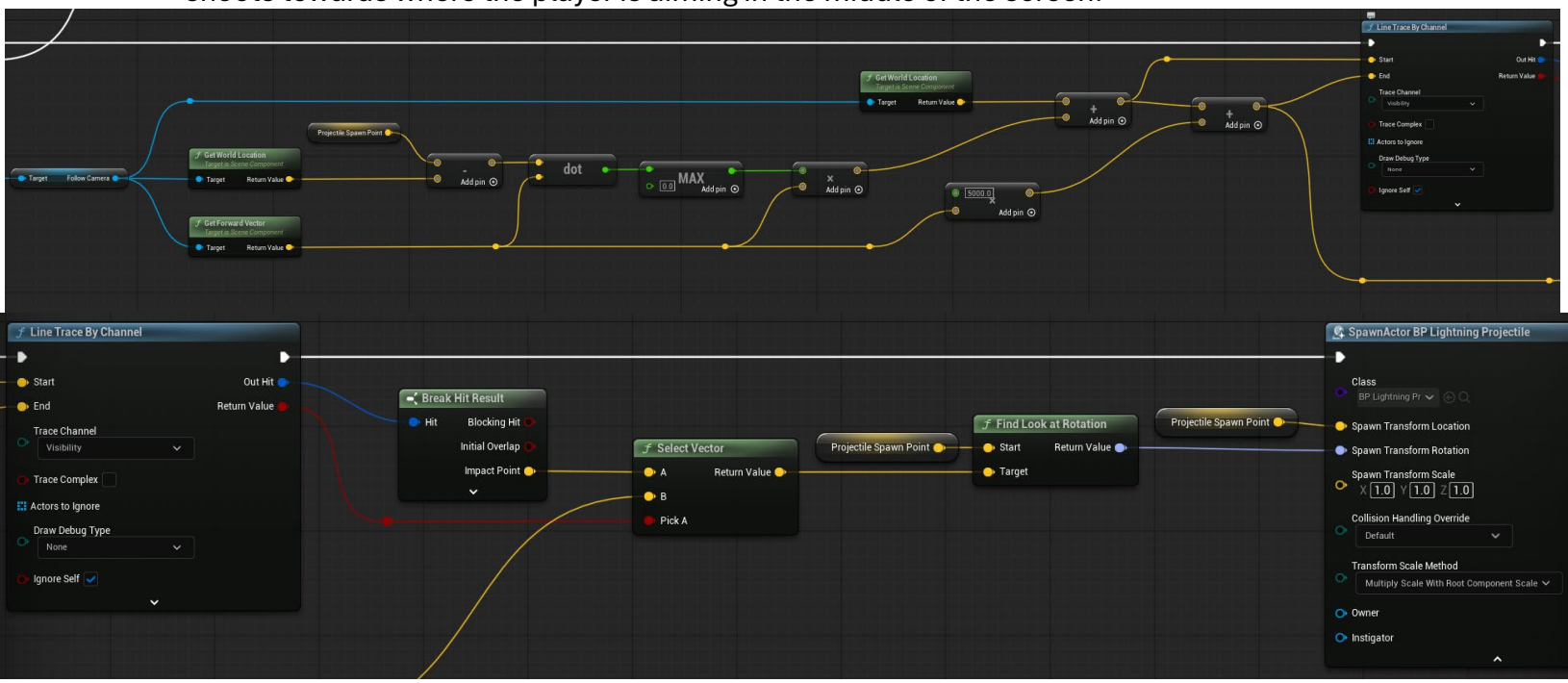


Then call the FireProjectile GameplayCue, and end the ability:



Actively Aiming

Use the camera's looking direction to set the projectile's rotation direction, making sure it shoots towards where the player is aiming in the middle of the screen.



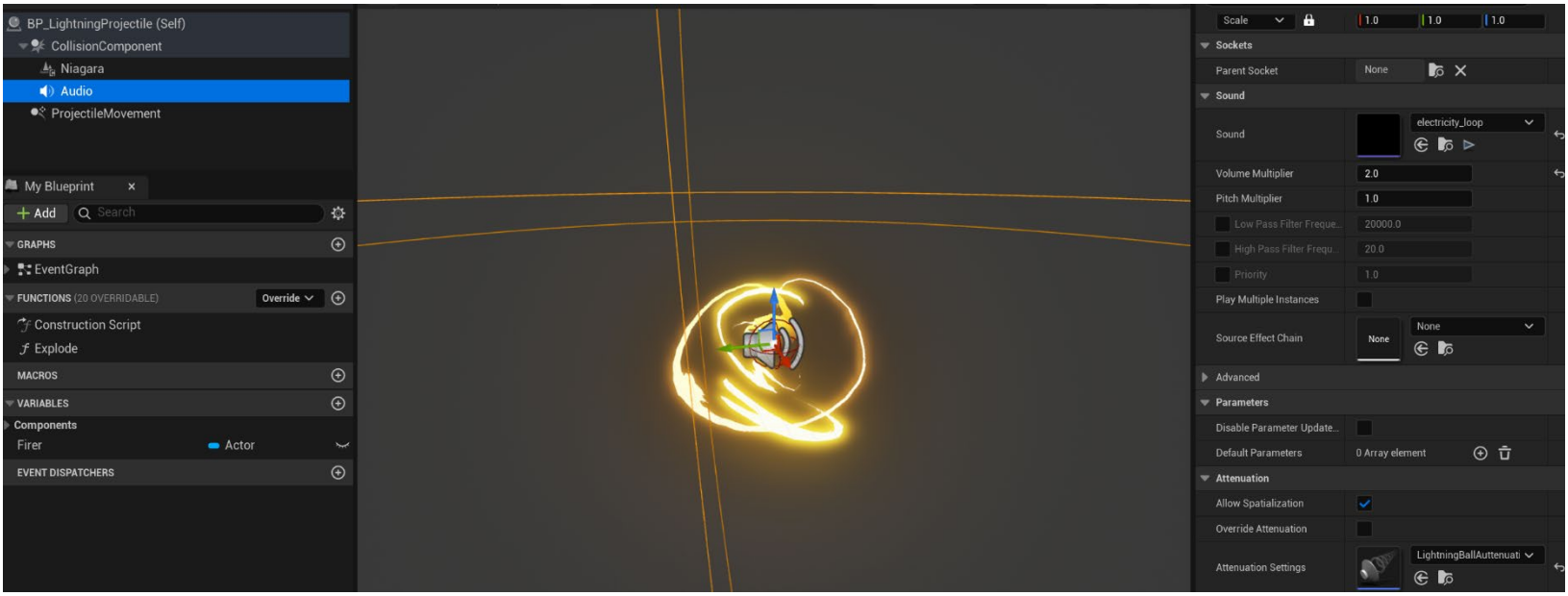
If the line trace hits something, use this position as the Target for the Find Look At Rotation. If it doesn't, use the end of the line trace.

At the end, do the same as the previous section. Tell the projectile who fired it, call the gameplay cue, and end the ability.

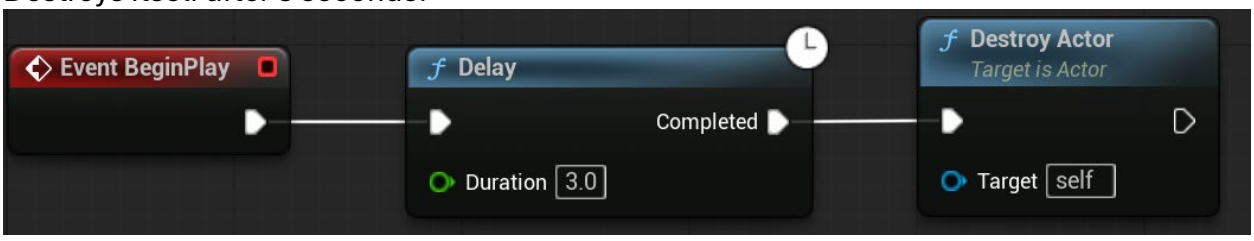
The Projectile

I decided to use a normal Actor instead of a Gameplay Cue Actor for the projectile because it uses physics to move and collisions to detect a hit. An actual object in the world with interactions, instead of a simple visual effect.

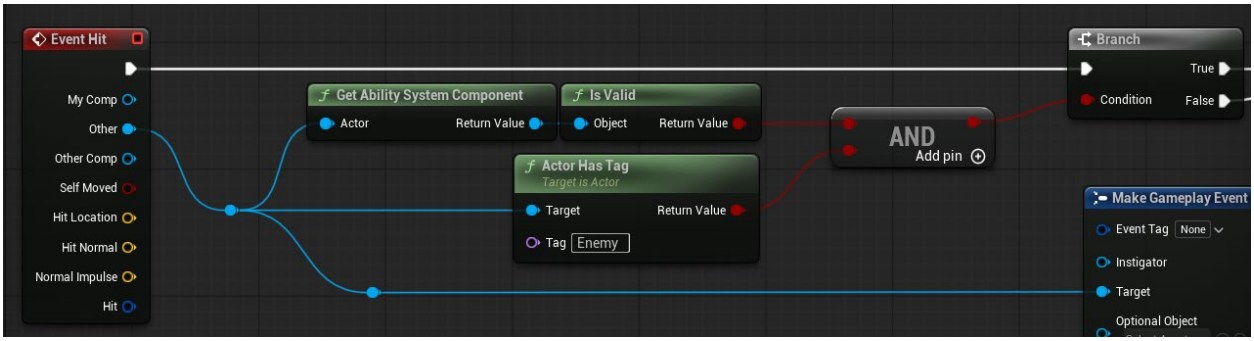
The projectile itself is an Actor with a Projectile Movement component, the Niagara effect, and a sound with attenuation:



Destroys itself after 3 seconds:

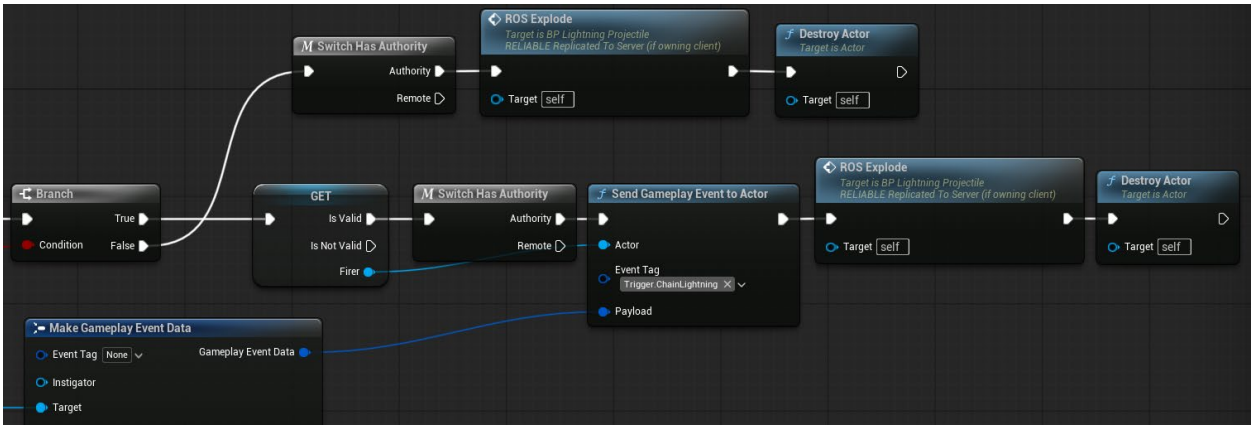


Upon hitting something, it validates if the hit actor has an Ability System Component, and if it is an enemy based on a Tag:

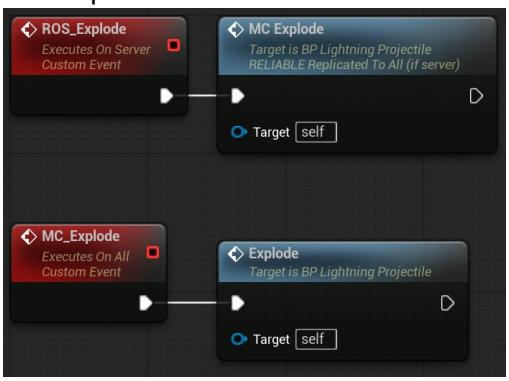


All enemies have both. I used a Tag since no functions are called on the enemy itself so it doesn't need a cast or an Interface.

If the projectile doesn't hit an enemy, just call Explode and Destroy itself.
If it is an enemy, send a Gameplay Event using the player that fired the projectile, and explode-destroy as well.



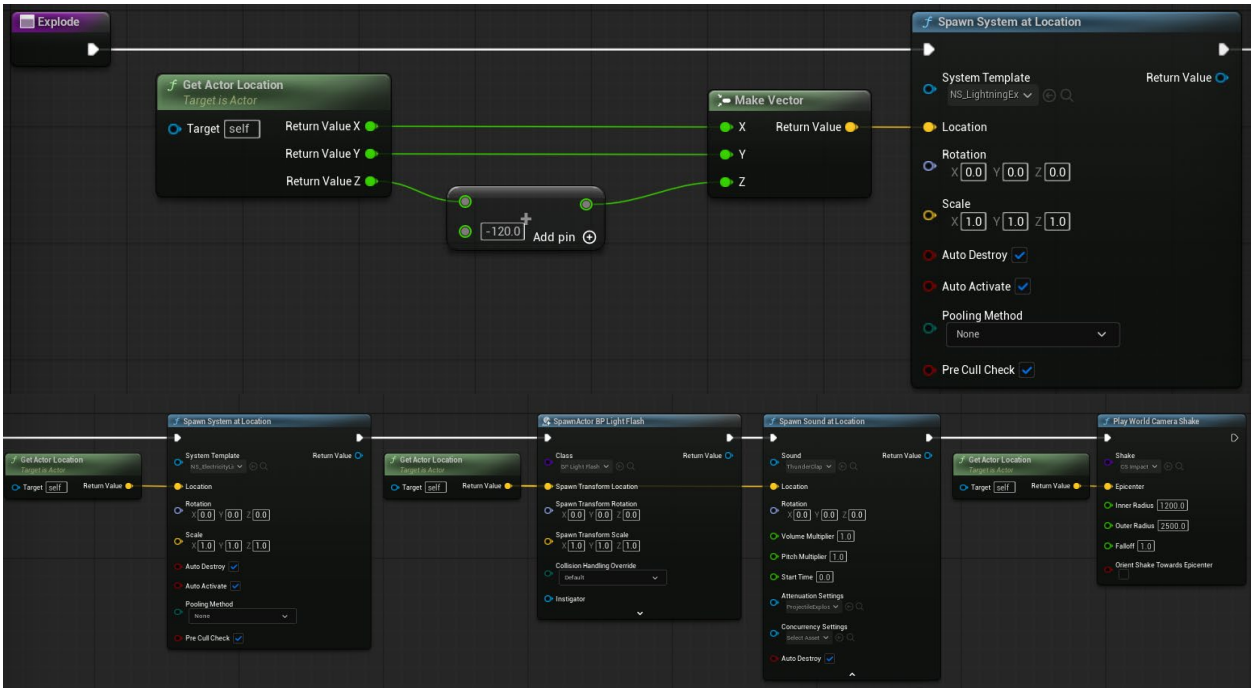
The explosion is called as a Run on Server Multicast event



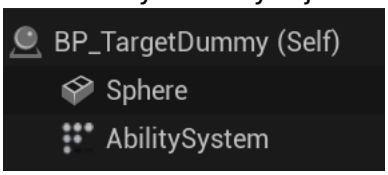
Which calls all of the effects:

- The explosion Niagara effect
- The lingering lightning effect
- The illumination effect
- The sound effect
- The camera shake

Read more details about it in the Effects section.

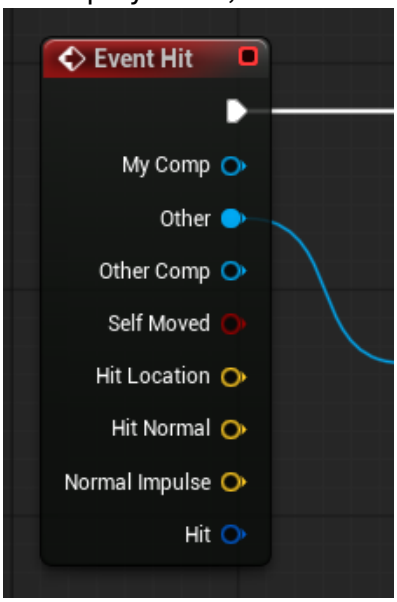


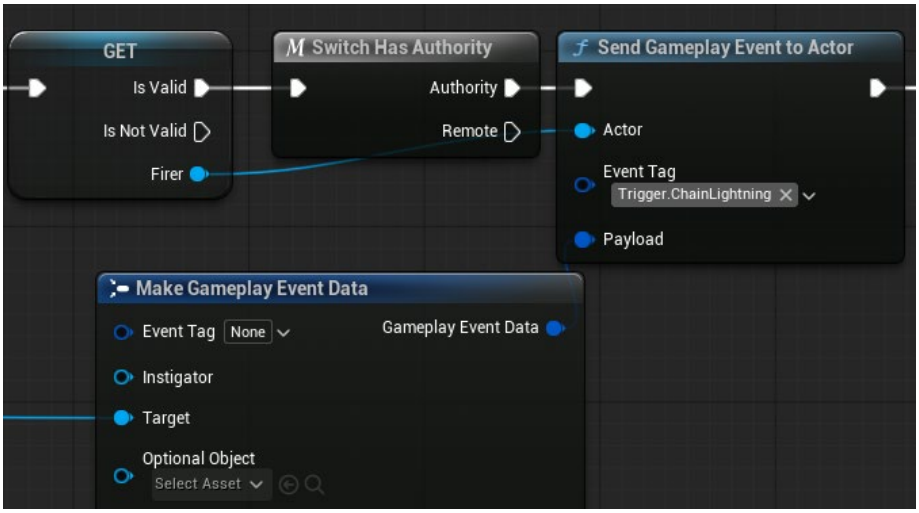
The Enemy Dummy is just a simple actor with the Ability System component:



Which, as mentioned before, is used as a validation in the activation of the GA_ChainLightning ability call.

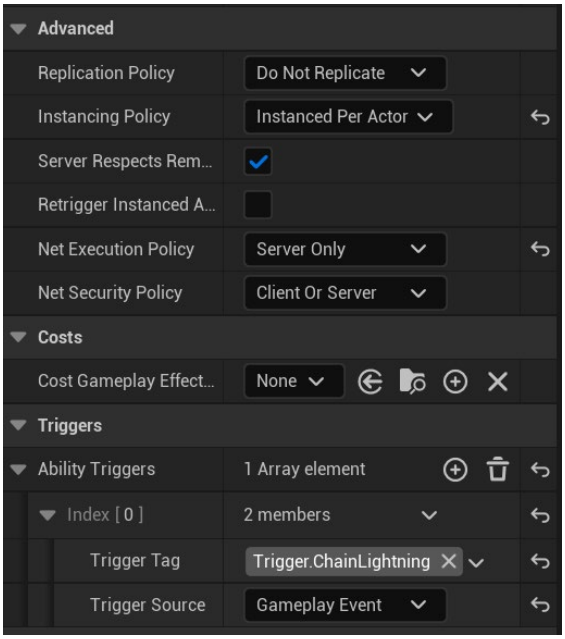
When the projectile hits, it gets the actor that was hit by it, and if it is a valid enemy, put it in the Target pin of the “Make Gameplay Event Data” that gets sent as the payload of the Gameplay Event, which is sent to the Player that fired the projectile:





The Chain Lightning Ability

Settings



It is **Instanced Per Actor** to make sure each player has its own chain of enemies to calculate, and no variables are shared within them.

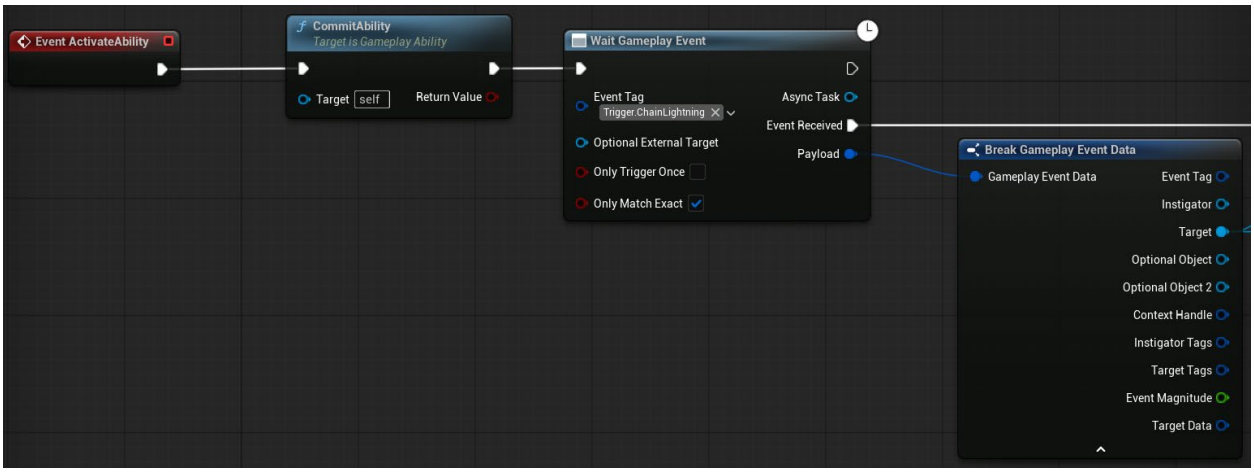
It is also **Server Only** since the Gameplay Event that triggers it runs only with authority, as well as all the variables that come from the attributes are only dictated by the server.

The Gameplay Cues are also fired by the server, so all clients can see them.

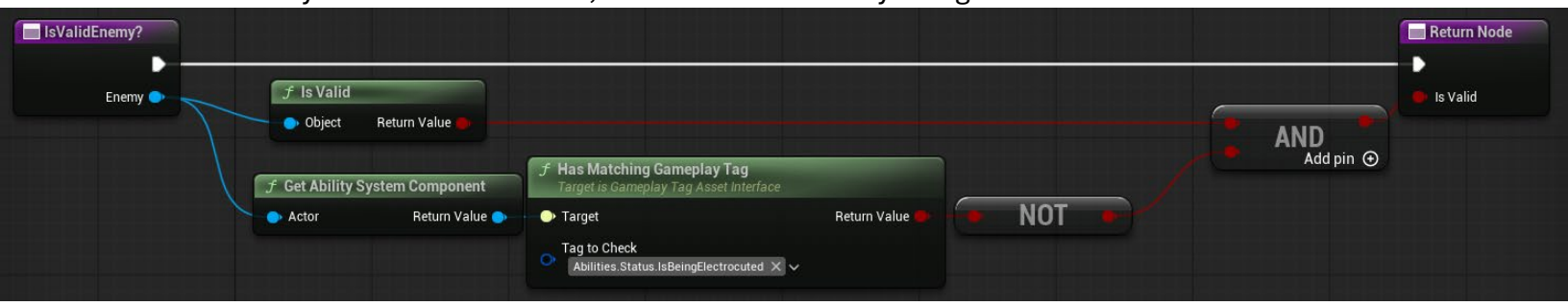
The ability gets activated by using the **Triggers**, and the **Trigger Tag** is the same as the one the projectile sends via the Gameplay Event.

The Algorithm

Commit the Ability, wait for the trigger, and get the payload where the enemy hit by the projectile is stored.



I created a **IsValidEnemy?** pure function to reuse in many places.
An enemy is valid if the actor is, and also not currently being electrocuted:



I use an array [**ActorsHitSoFar**] to keep track of all enemies that were successfully hit by the lightning during the execution of this ability.

So, first thing I do after validating the enemy, is clear this list just in case, and add the first hit enemy to it, as well as applying the damage to the enemy:

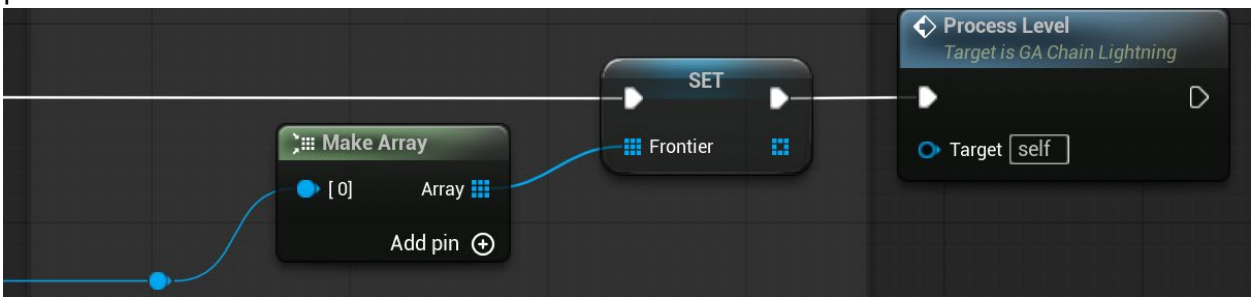


The application of damage is a visual effect, so go to that section for more details.

My algorithm works by using a breadth-first-search approach, where each bounce goes through each enemy hit in the current loop level, instead of using recursion.

For this, I use a variable called **Frontier**, which will hold every enemy to be hit by the lightning in the current level.

I set the enemy that starts the chain as the current only enemy in the Frontier, and start the process:

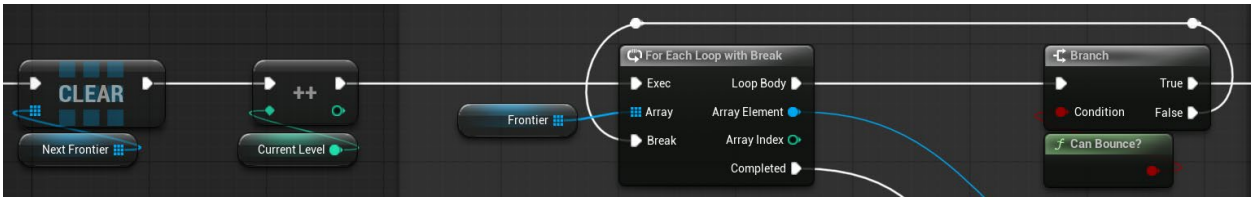


Another variable called **Next Frontier** will hold all the enemies that will be processed in the next level.

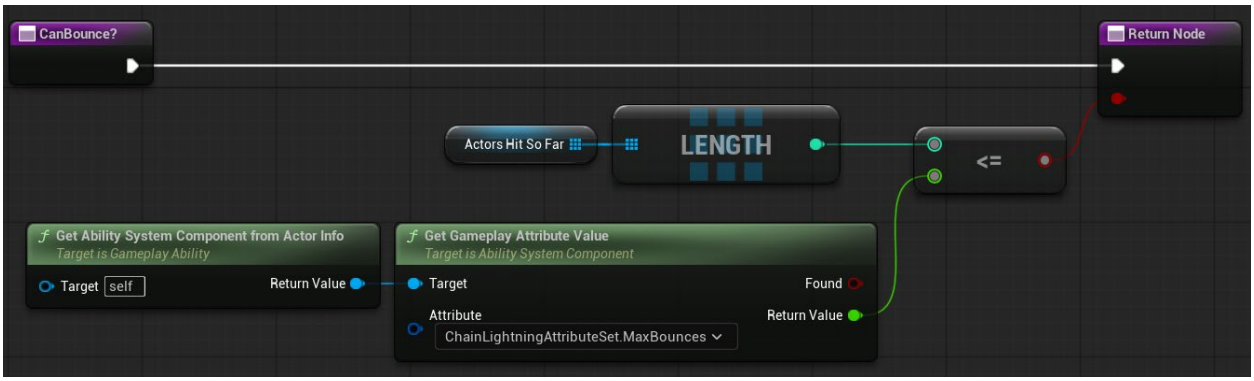
First of all, I clear the **Next Frontier** variable, since the Frontier variable is holding the current enemies to be processed.

I also increase the level **Count**, making the first enemy hit correspond to Level 1.

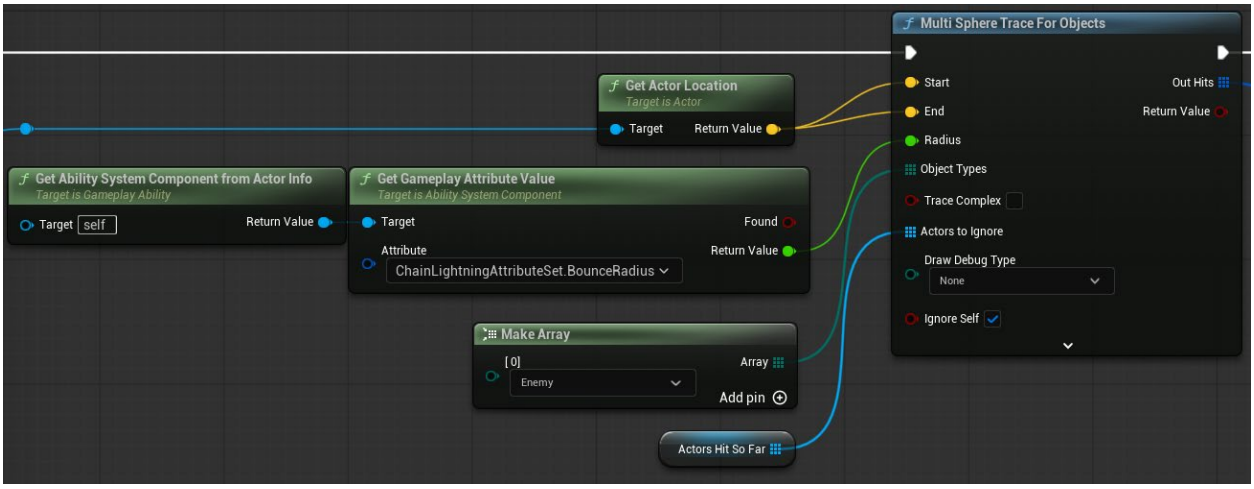
The loop starts, iterating the **Frontier** elements.



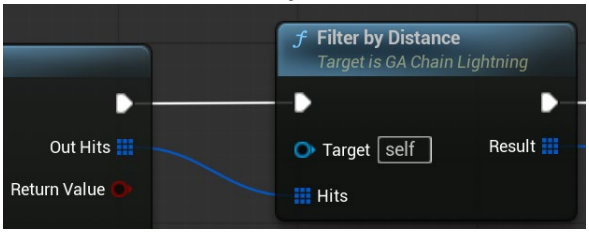
Determine if we can still bounce to the current enemy.
Which is a validation where if the actors hit so far are <= the **Max Bounces** allowed:



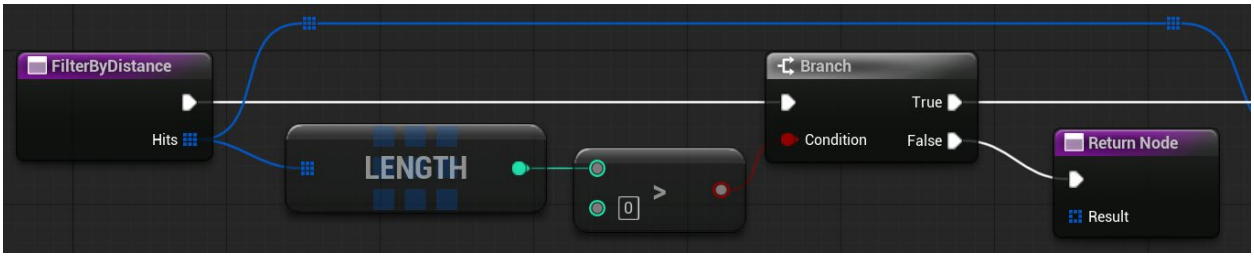
So now, I use a **Multi Sphere Trace** by Object type to detect all of the enemies in the **Bounce Radius**:



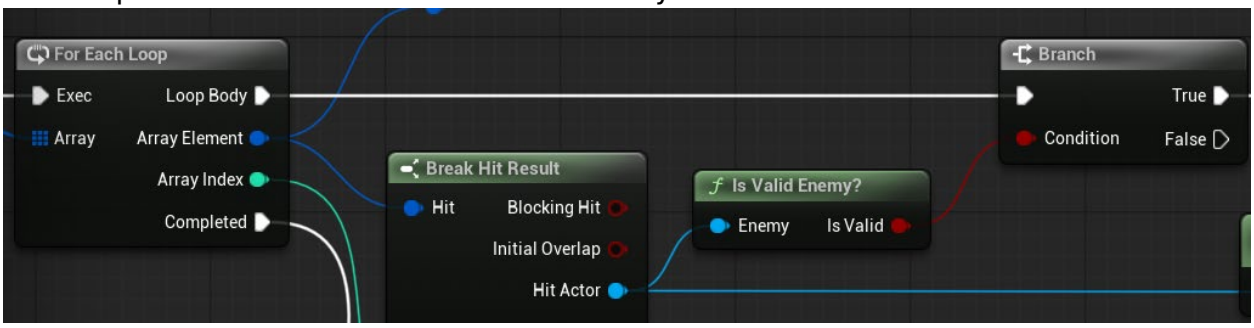
Then I filter the hits by distance, from shortest to longest:



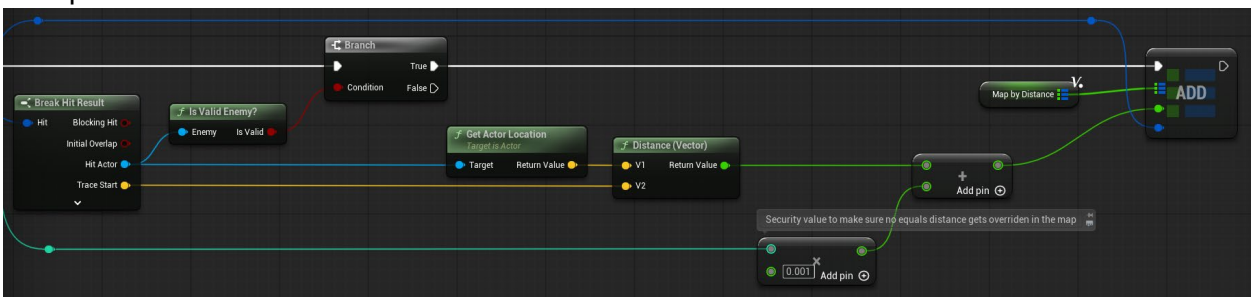
Here's how it works:
First, make sure there were hits



Now loop them and make sure it is a valid enemy.

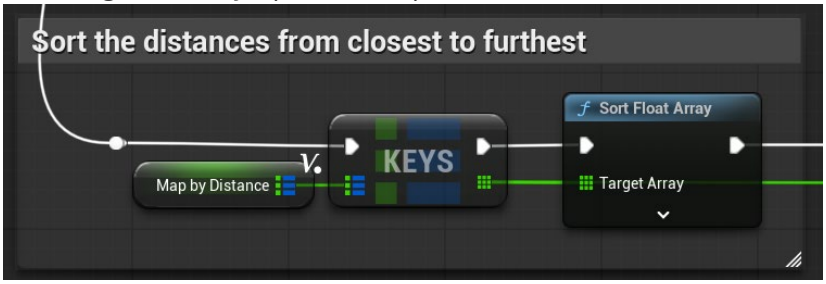


For each enemy, get the distance to the source enemy's location (Trace Start), and add it to a Map that associates each distance with the Hit Result.

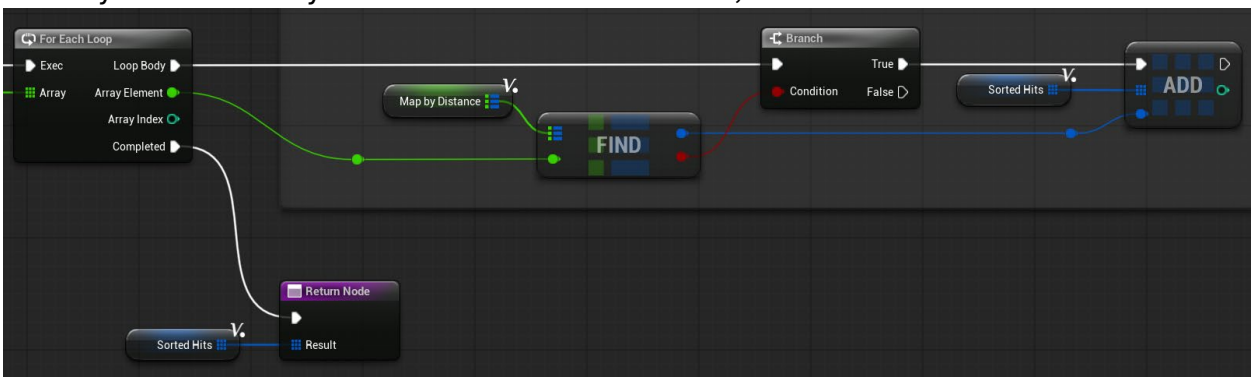


Each distance added to the map has a very small Index * 0.001 added to it since identical distances would get overridden. To prevent that, this security number is added so all distances are unique.

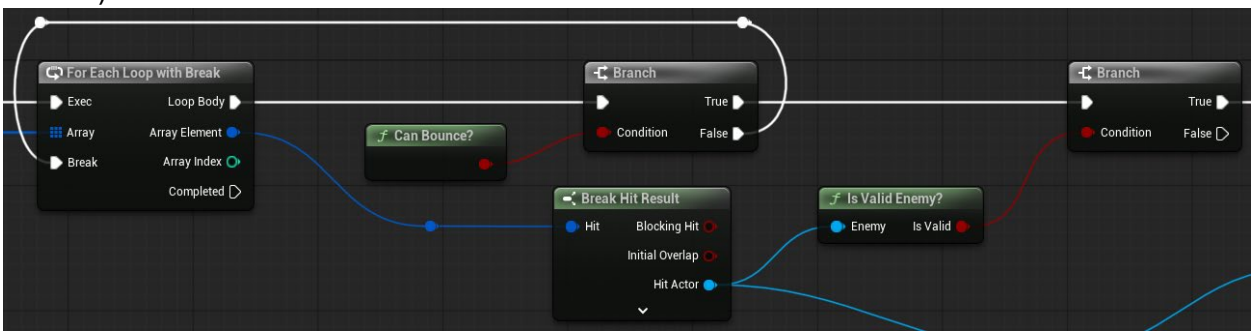
Then, I get the keys (distances) and sort them:



And with this, iterate the sorted distances, get the associated Hit Result, and add them in an array. This new array will contain all the Hit Results, sorted from closest to furthest:



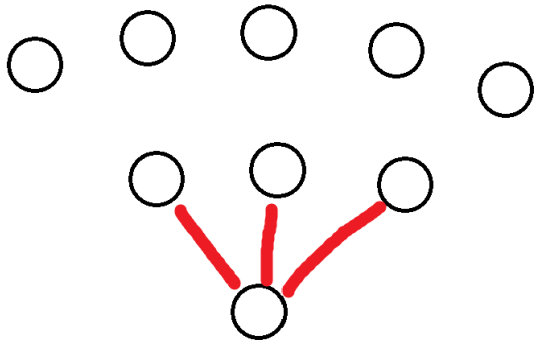
Once sorted, loop them (always making sure that we can still bounce, and that the enemy is valid):



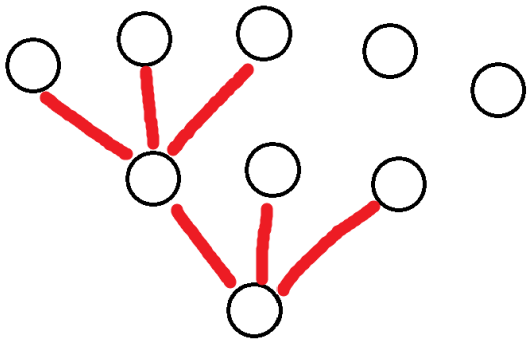
Now here comes a very crucial part of the algorithm.

If we just iterate the enemies in the Frontier as is, and create a chain link towards itself and the enemy that called the Trace, the first enemy in the frontier would claim all of the links to every close enemy to itself in the next frontier level.

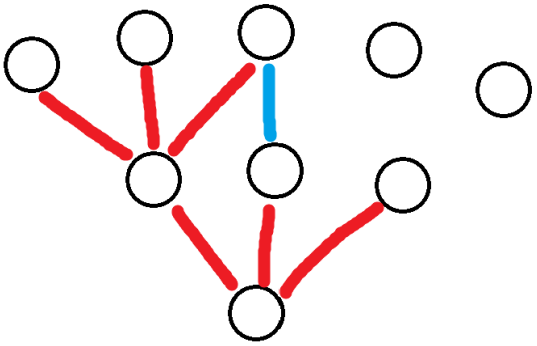
Here's an example, let's say we have this formation of enemies, with a **Chain Amount** of 3, and we are in level 2 of the chain:



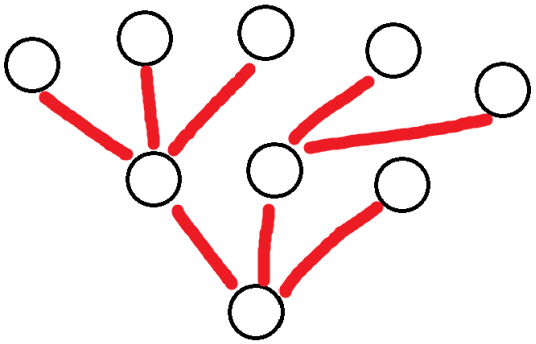
Now the level 2 chain starts connecting enemies. If we just allow each enemy to make a connection with all valid enemies as is, the current enemy in the loop would be able to claim a connection to an enemy that is closer to another yet not iterated enemy in the current level. Like this:



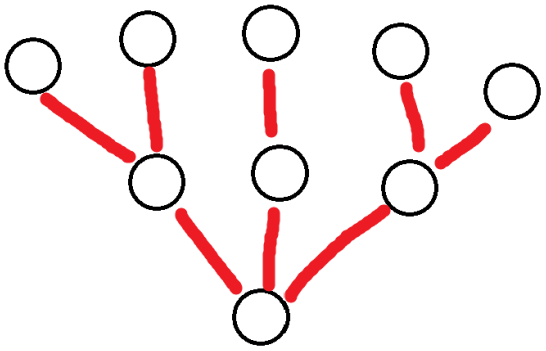
And as we can clearly see, there is a shorter connection that the next enemy in the loop should have claimed:



And the final chain of lightning would look like this:

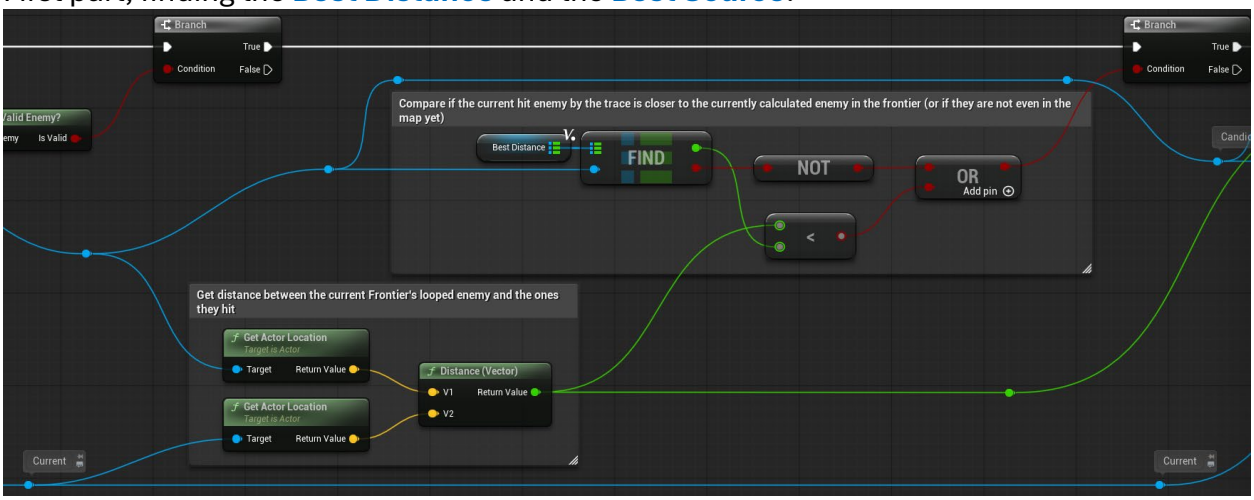


Which looks weird. It should look something like this:



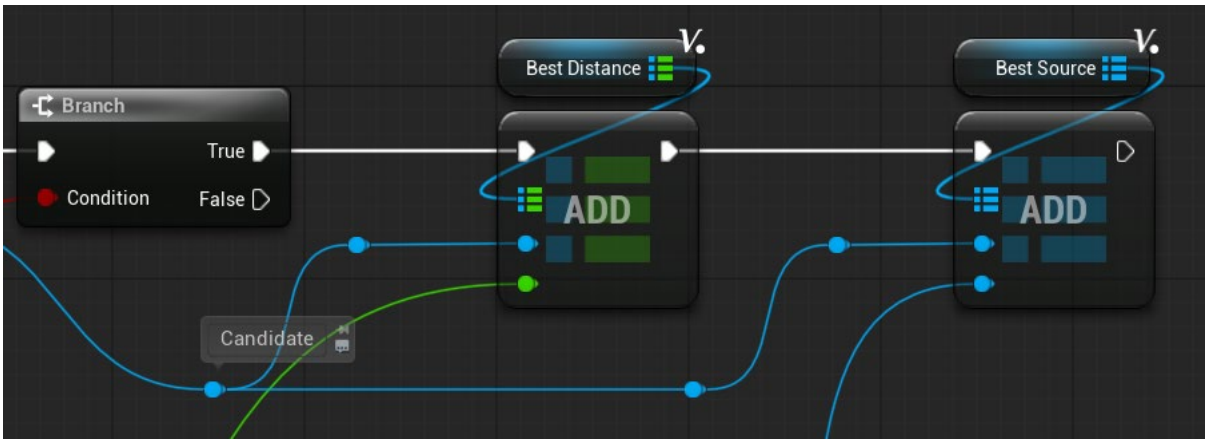
That's what this part of the algorithm achieves: That every chain connection is the smallest available for any given enemy in the chain, no matter their ordering.

First part, finding the **Best Distance** and the **Best Source**:



We need to compare each enemy in the potential next frontier with every enemy in the current frontier, and know which one has the closest distance.

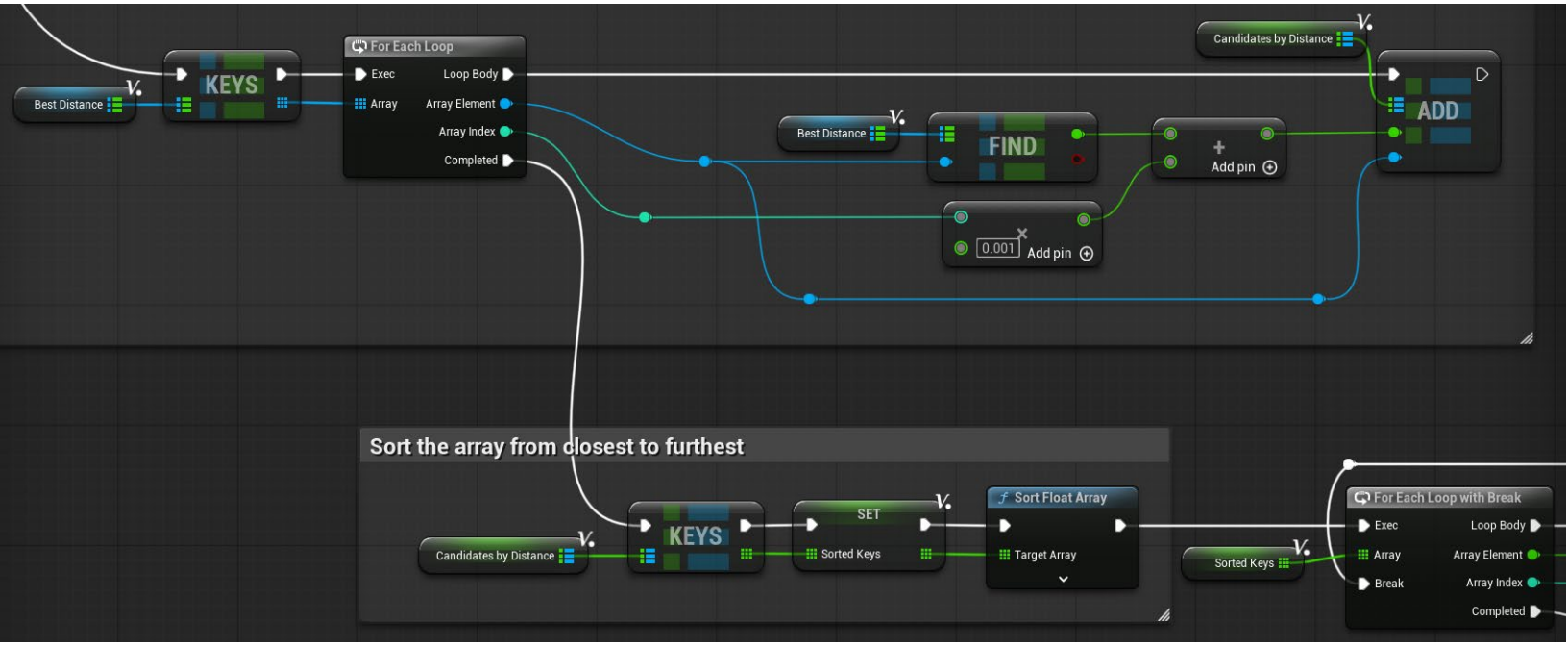
Compare every enemy hit by the traces (**Candidate**) with every enemy in the Frontier.
If this Candidate is not on the map, or if they are and we have found a shorter distance for them, replace it:



Save the currently found best distance (**Best Distance**) and what enemy produced this best distance (**Best Source**).

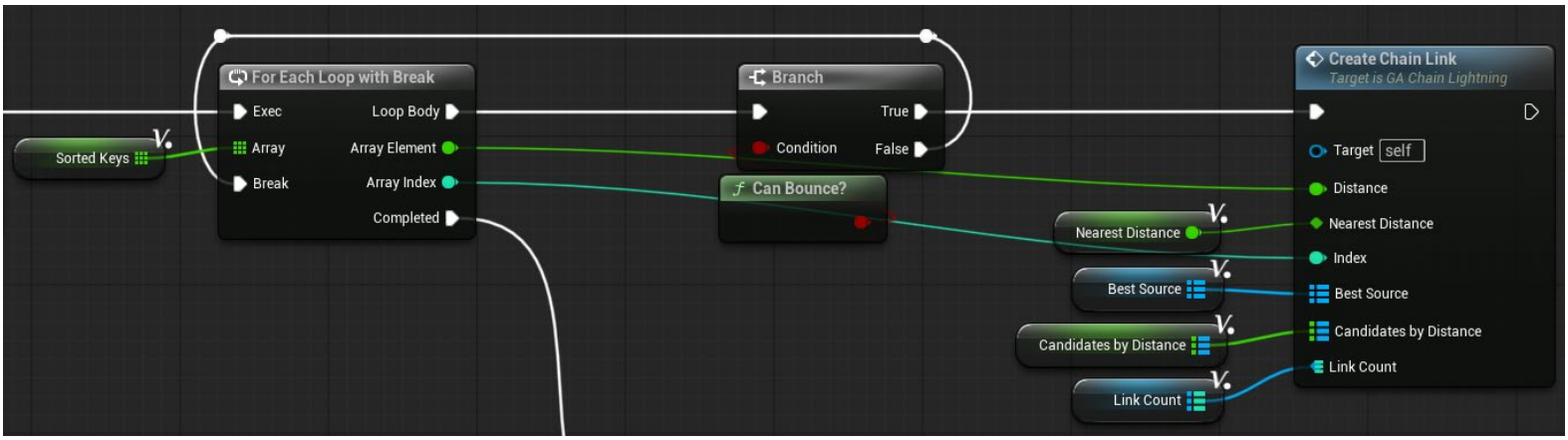
We do this for every enemy in the **Frontier**, and so at the end, we get every enemy in the **Next Frontier** associated with what enemy in the current Frontier is the best one to make a chain with.

Now I need to make the shortest connections to a source enemy first. So I sort the distances using the same method as in the FilterByDistance function:

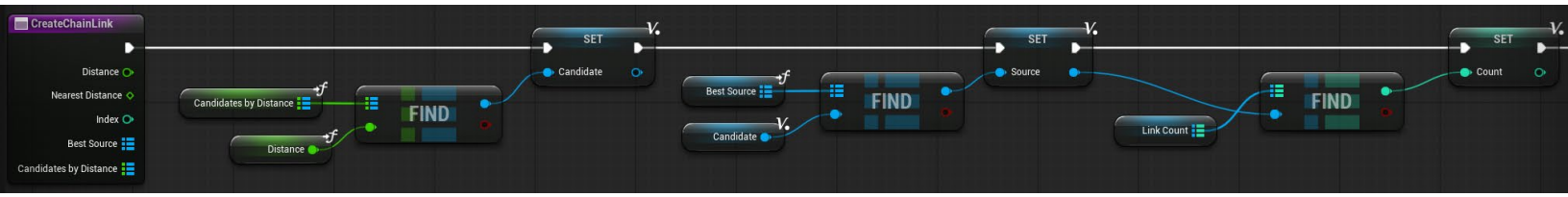


Candidates by Distance now holds every unique Candidate that was hit by every enemy in the current Level Frontier.
So, by sorting them, we make sure every chain is made from the shortest one first, no matter the order the Traces hit the Candidates.

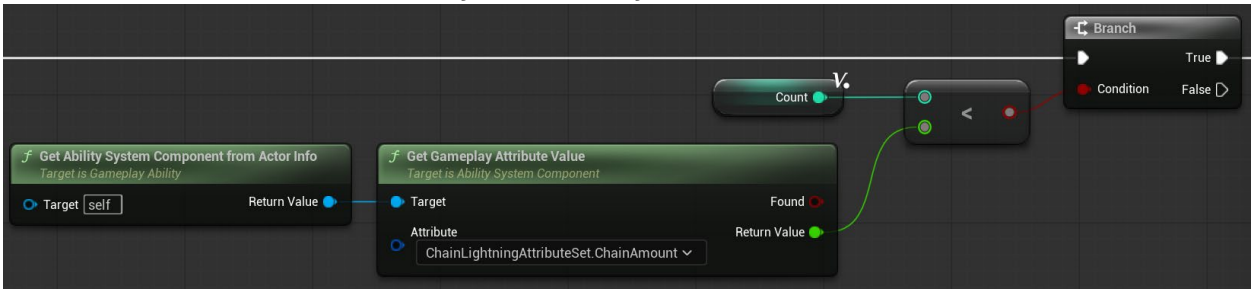
And now by iterating the sorted distances, we create the chain links:



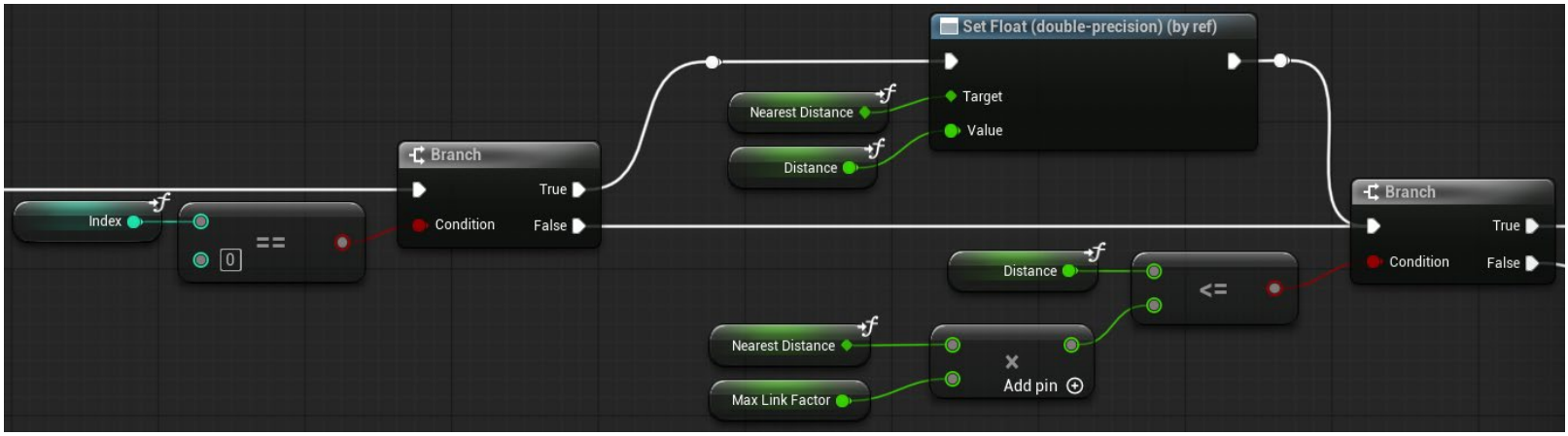
First of all, I get the current **Candidate** and **Source** enemy associated with the currently iterated distance. Then register this chain in another map called **Link Count**, which stores the number of chains connected to a given source enemy through the ability.



With it, we validate that the enemy creates only the allowed number of link chains:



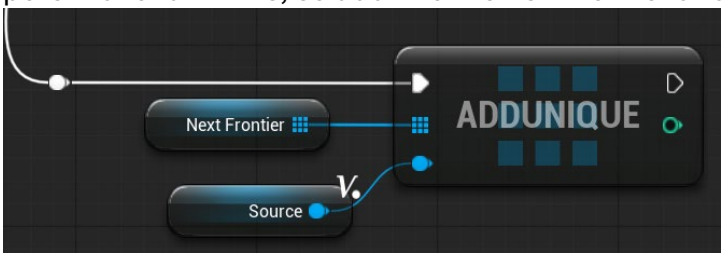
If the chain is already at max, skip this Candidate. If it is within limit:



And a **Max Link Factor** multiplier variable, which will be multiplied by the **Nearest Distance** and compare it with the current distance being iterated, if it is <= to it:

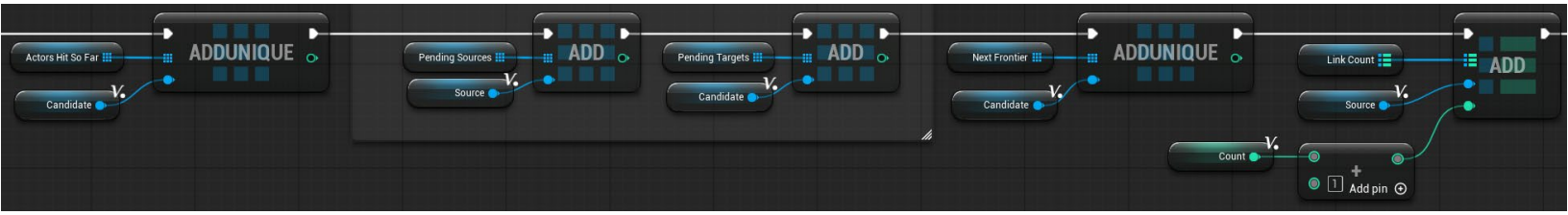
This makes it so distances that are too long (larger than Nearest * Multiplier) are not considered for the chain. Essentially an aesthetic validation, which is based on the first enemy in the list. That's why I added "index == 0", storing this as the shortest distance, and comparing it with the multiplier each time.

If the Candidate doesn't qualify, it means that it is within the chain amount, but ran out of potential chain links, so add it to the next frontier and try the next round:

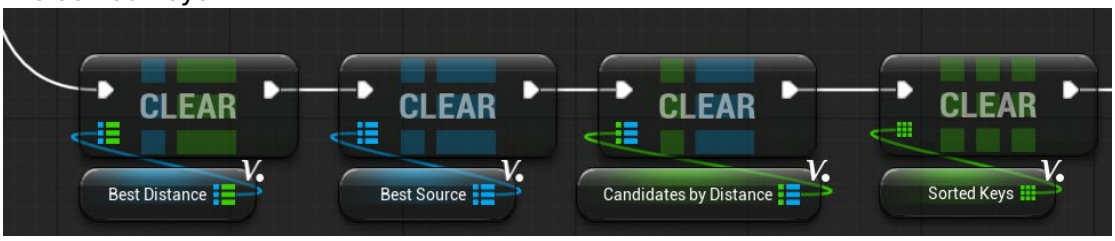


If the Candidate does qualify:

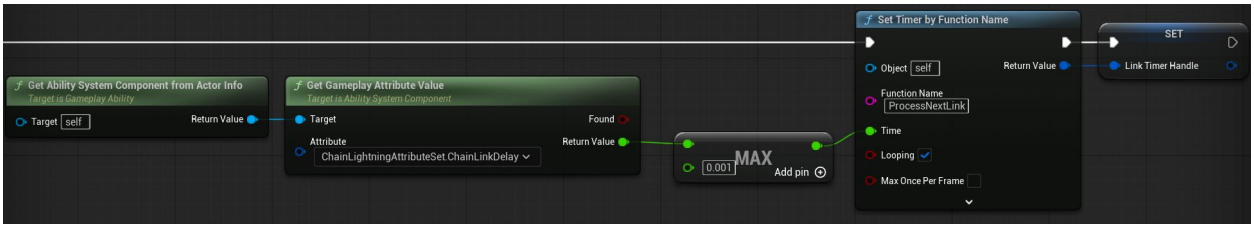
- Add it to the **Actors Hit So Far**
- Add it to the Pending Lists
    - **PendingSources** and **PendingTargets** are variables that will be explained later
- Add the **Next Frontier**
- Increase the chain count for the current source



Once all enemies have been assigned a chain link for this Frontier, clear all the maps and the sorted keys.



And now I call a Timer by Function Name, which adds an optional delay for each chain linking:



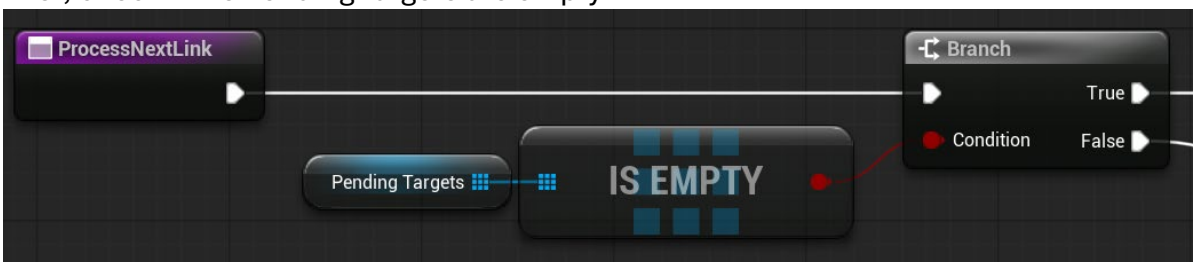
The timer is the bigger value between the ChainLinkDelay or 0.001 seconds.
I store the timer in a variable because it's going to be cleared soon.

The purpose is to create a fully controllable lightning, where there could be a delay not only for the bounces between whole levels, but also between each chain linking.
This makes it so in order to process the next frontier, it must wait for all chains to connect first.

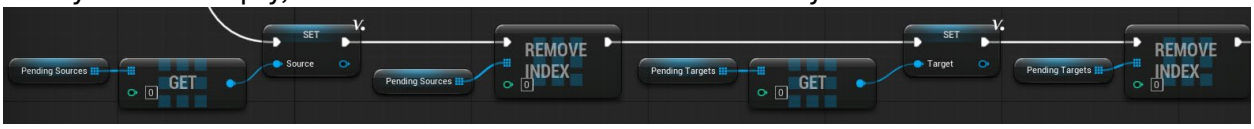
This is where I use the **PendingSources** and **PendingTargets**.

Personally, prefer this approach with just minimal chain delay, since it gives the player more travel feedback for each lightning link, as well as creating a crisp sound effect cluster, since each enemy is electrocuted at a slightly different time.

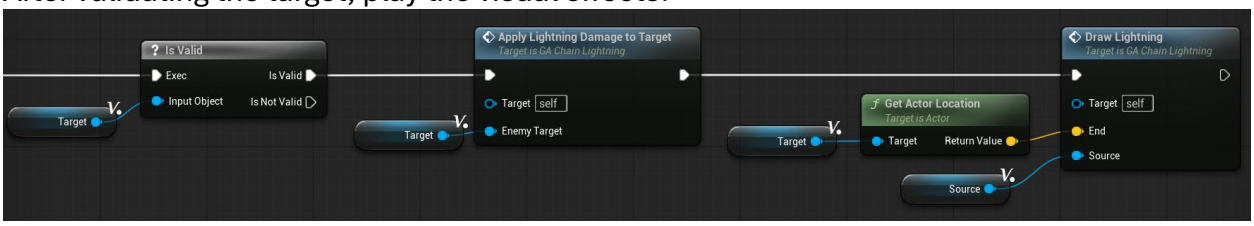
First, check if the Pending Targets are empty:



If they are not empty, take the first element out of each array:



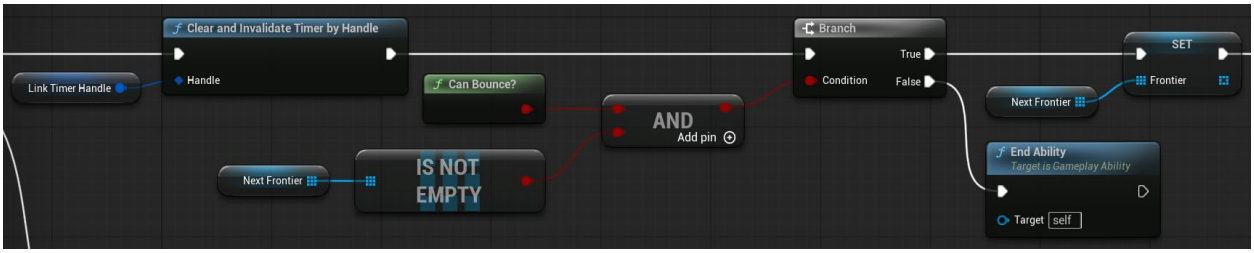
After validating the target, play the visual effects:



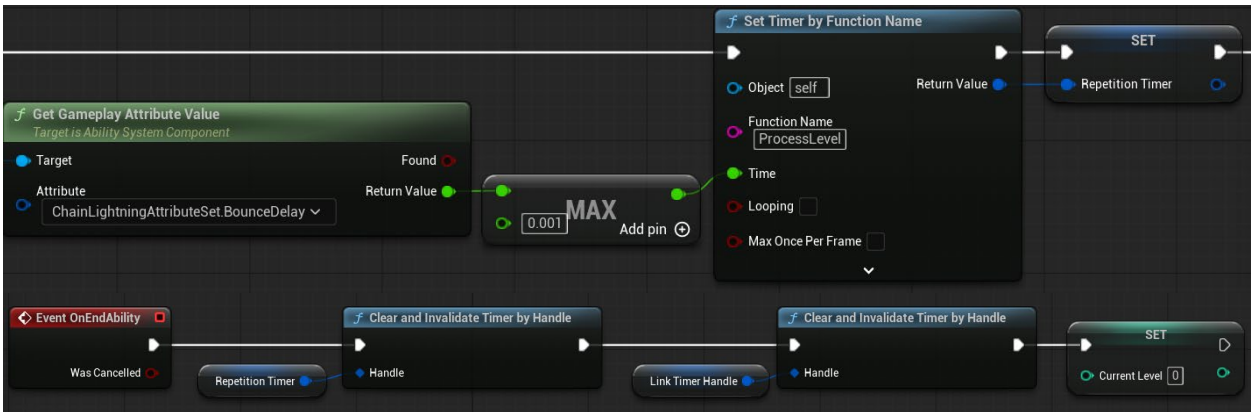
And so, the functions will be called via the timer, and play the effects one by one, until there are no more enemies in the array.

Then I clear the timer, and make sure that the **Next Frontier** is not empty and that we can still bounce.

If not possible, the ability ends, and if we can continue, I set the **Next Frontier** as the current one:



And finally, call the **ProcessLevel** function again after the Bounce Delay, save the timer handle, and clear all timers upon Ability End.



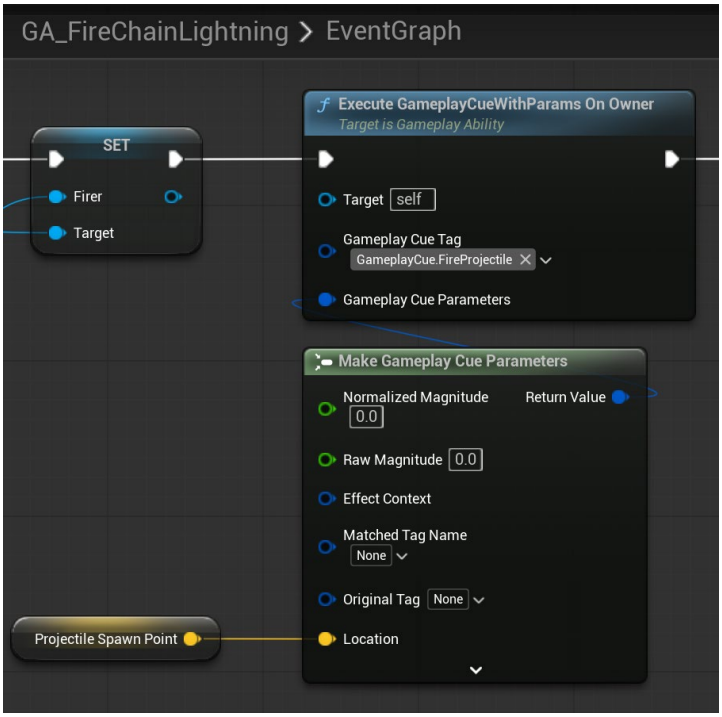
Visual and Sound Effects

All visual effects, except for the projectile, are Gameplay Cues, so all players can see them.

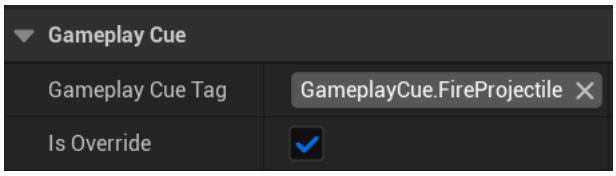
GC_FireLightning

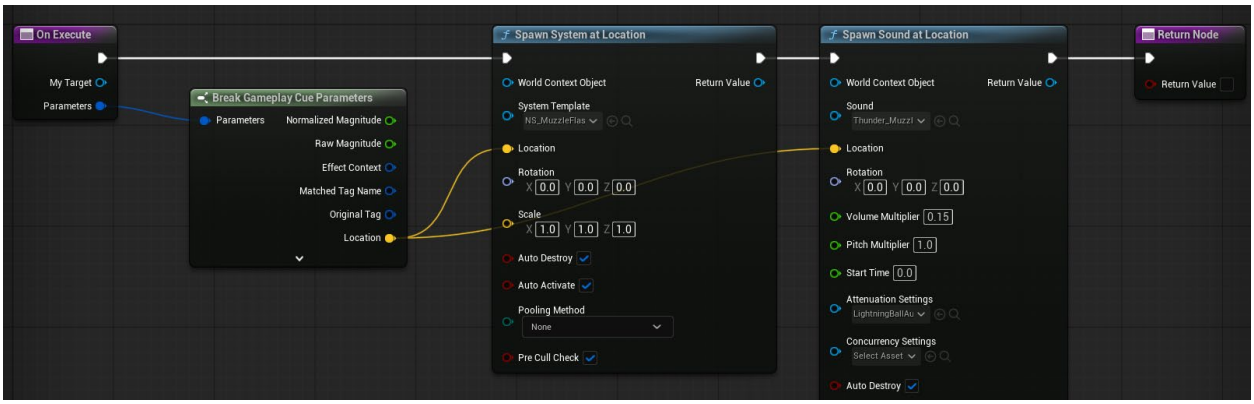
The visual electricity that happens at the player's hand location when firing the projectile.

Inside the GA_FireChainLightning, after the projectile is spawned, trigger a Gameplay Cue for the visual effect of the hand firing, like a gun muzzle, as well as a sound effect.



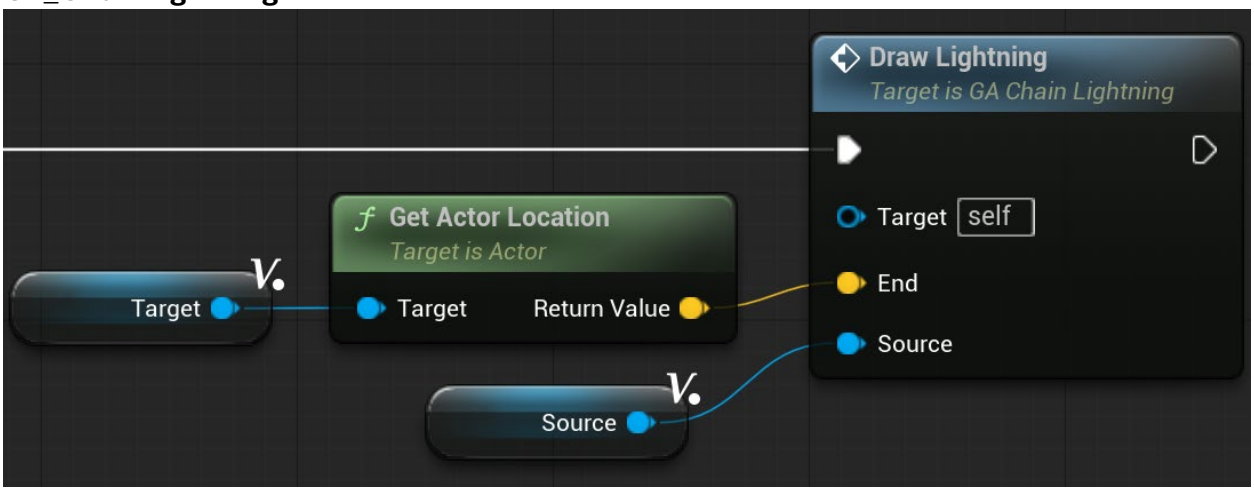




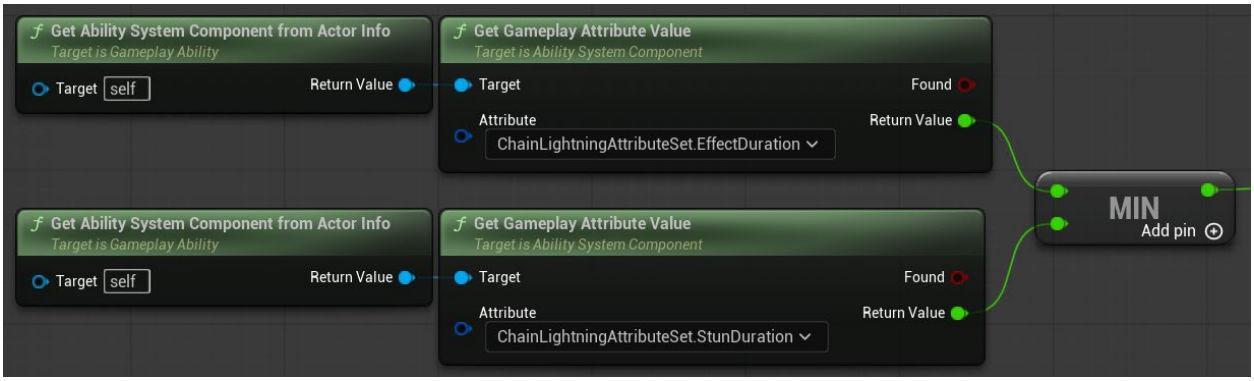


GC_LightningLink

This is the lightning chain effect that goes from one enemy to the other. Triggered by the **DrawLightning** function call inside the **ProcessNextLink** function in the GA_**ChainLightning**:

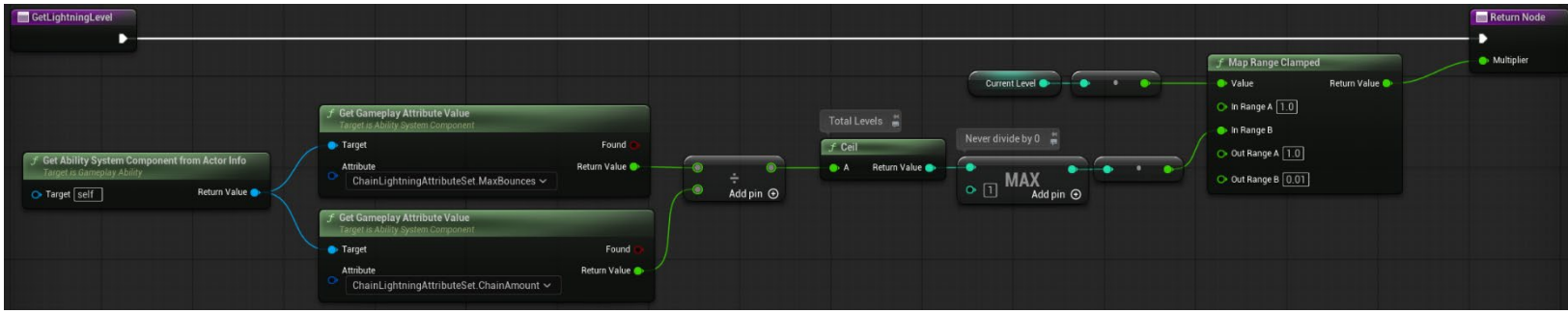


As a security measure, I make sure that the duration of this effect is the lower value between the **EffectDuration** attribute (the intended value) and the **StunDuration** attribute, which is meant for the electrocution effect.



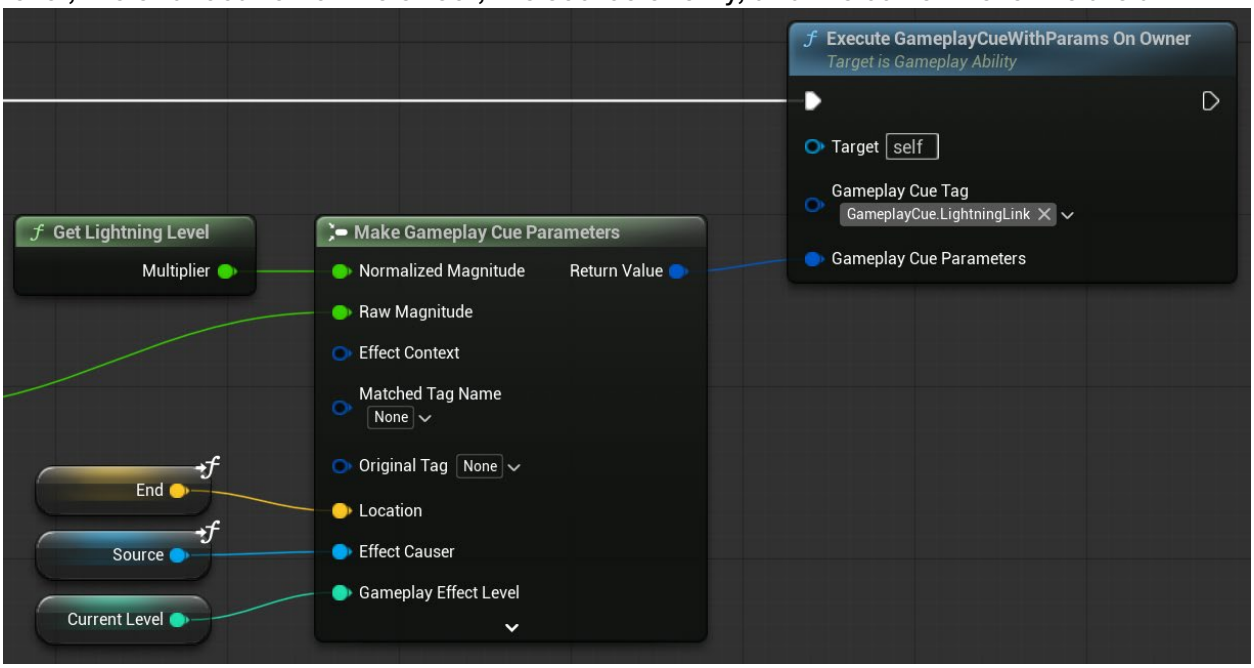
I designed the lightning chains to be *stronger near the start of the chain, and get weaker as it travels down the connections*.

I made a pure function called **GetLightningLevel** that maps the strength of the effect based on the current level of the chain.

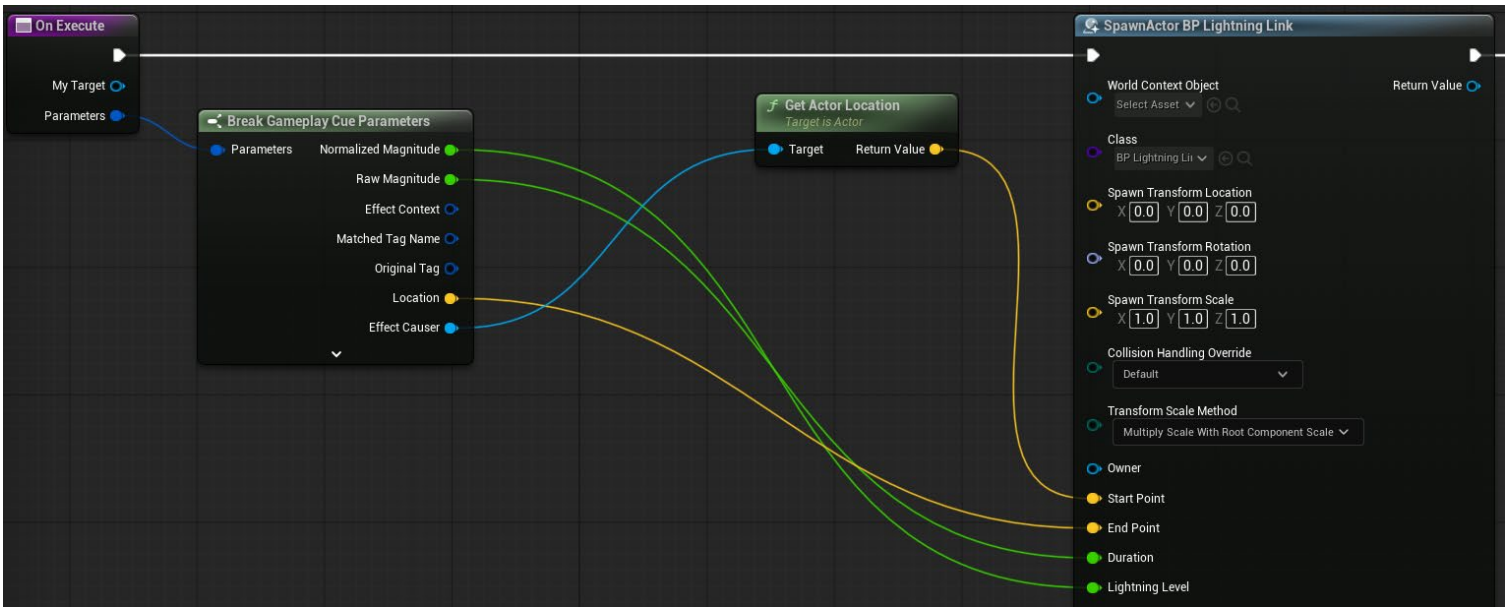


It calculates the number of levels the chain is going to have, and uses the current level to return a number between 1 and 0.01; this number is used to multiply the Niagara effect of the Lightning Chain.

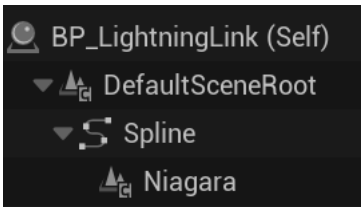
I call the GE, using the Make Gameplay Cue Parameters to pass the values of the lightning level, the end location of the effect, the source enemy, and the current level we are at:



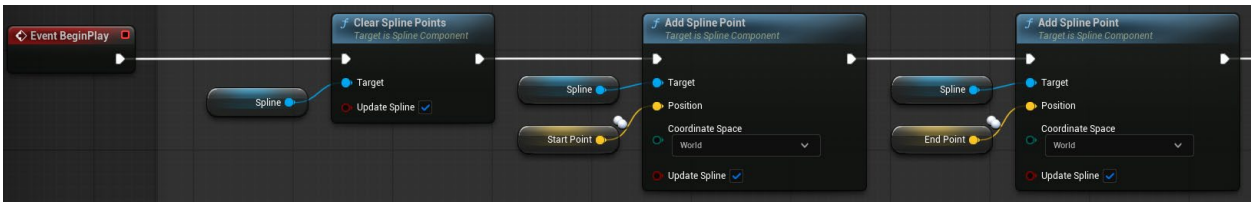
And so, overriding the **On Execute** of the **GC_LightningLink**, I spawn the effect actor:



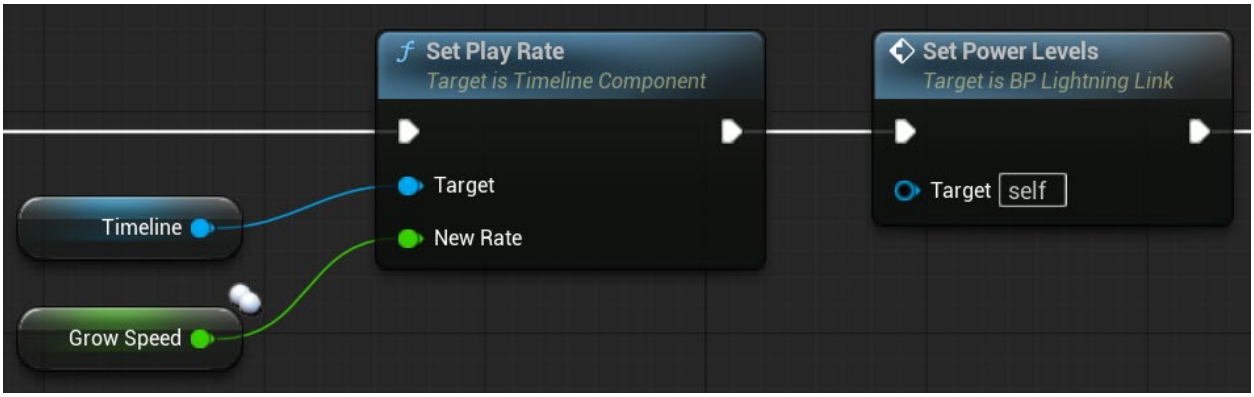
It has a Spline with the Niagara effect on it:



Using the parameters sent in the spawn, I set the Start and End positions of the spline:



Right after, I set its visual settings:



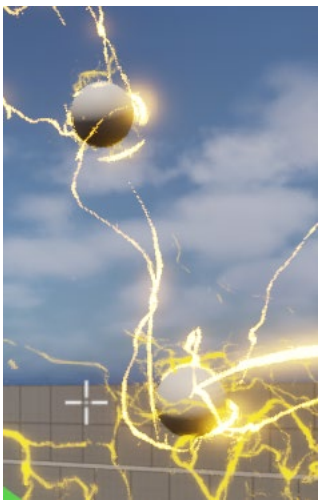


In **SetPowerLevels**, I multiply the default values by the float provided by **GetLightningLevel**, so the default value is when the effect is the most powerful, decreasing as it travels through the enemies.

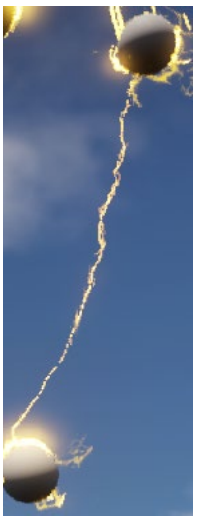
The parameters include (not limited to):

- Jitter
- Main Power Effect
- Brightness
- Density
- Width of the Lightining

Max Power:

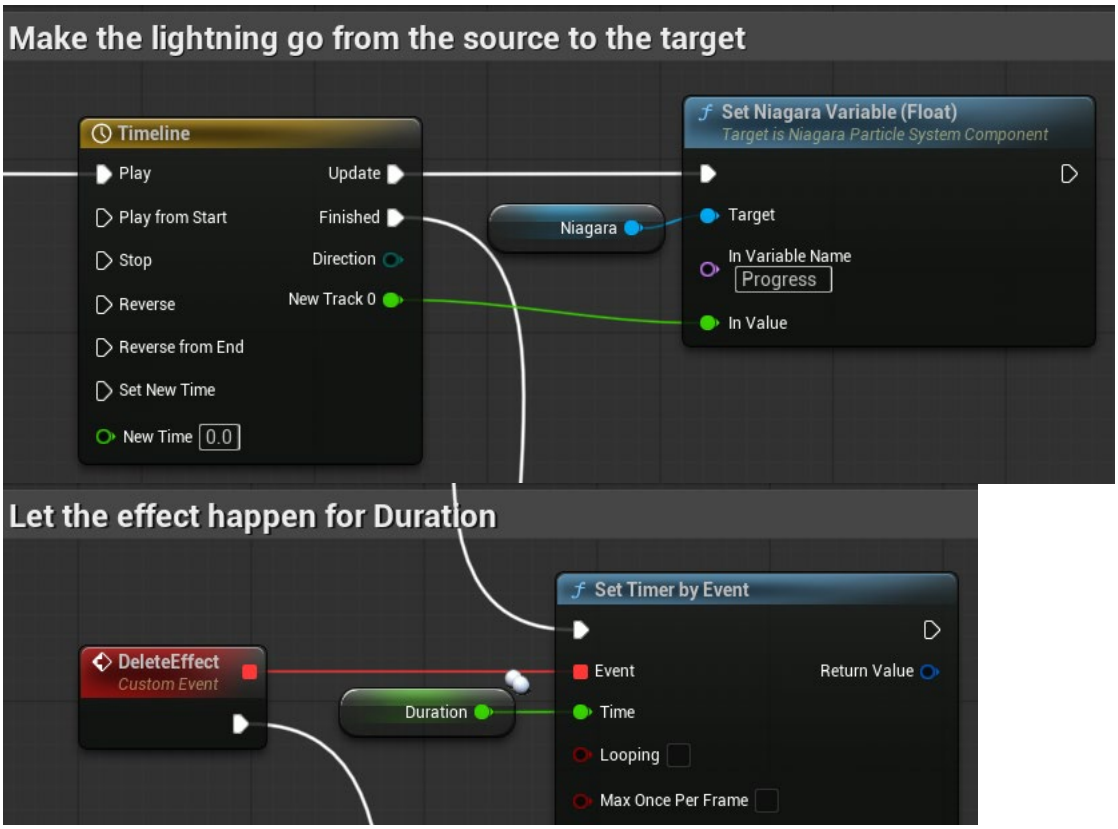


Min Power:

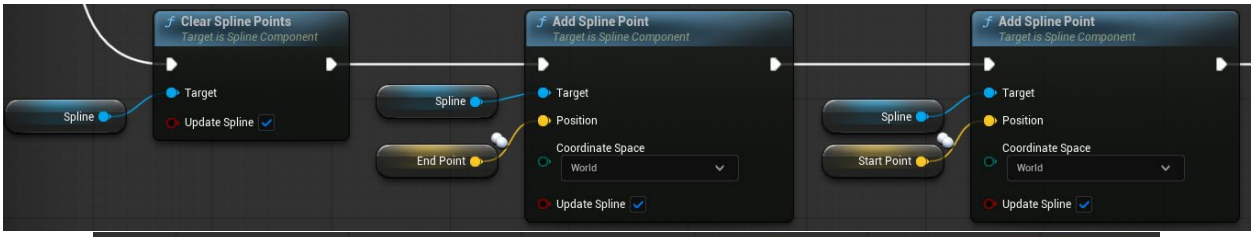


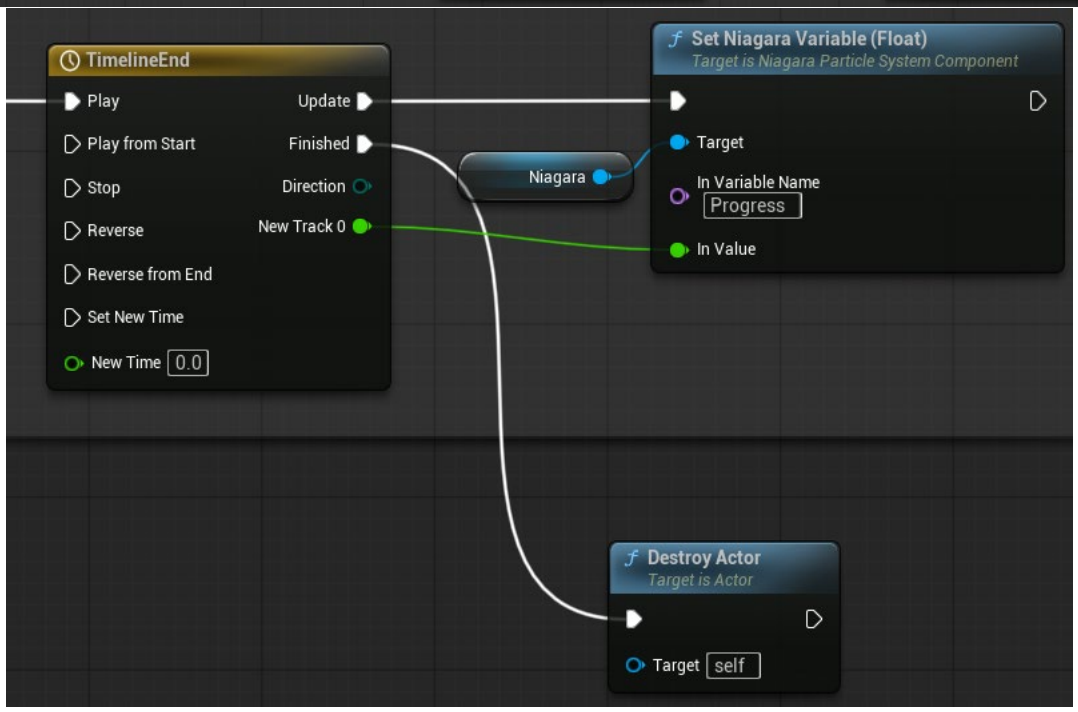
The Effect includes a “Progress” variable, which indicates how far along the spline it is located.

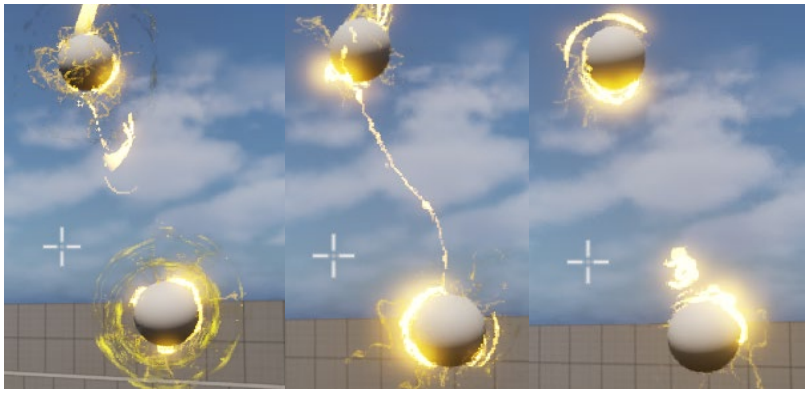
Using a Timeline that goes from 0 to 1, it will travel from the Start to the End points; and I let it stay like that for the **Duration** sent by the Attribute.



Once that is done, I switch the Start and End points, and play the timeline again, so the lightning travels from the Source to the Target before ending the effect, making a full travel from enemy to enemy.

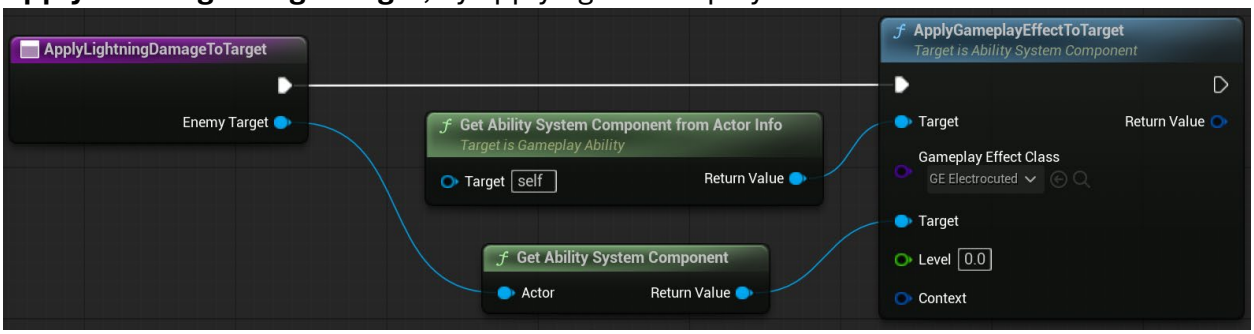




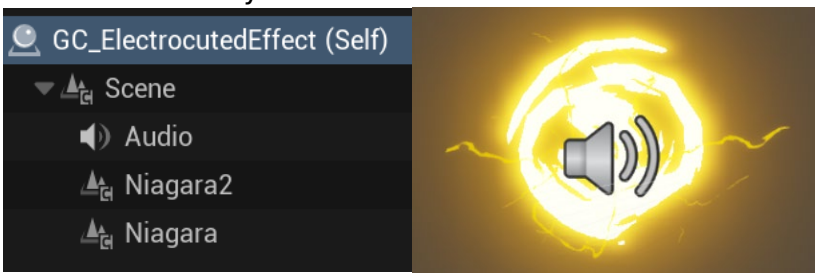


GC_ElectrocutedEffect

This Gameplay Cue is called from the **GA_ChainLightning** as well via **ApplyChainLightningToTarget**, by applying a Gameplay Effect:

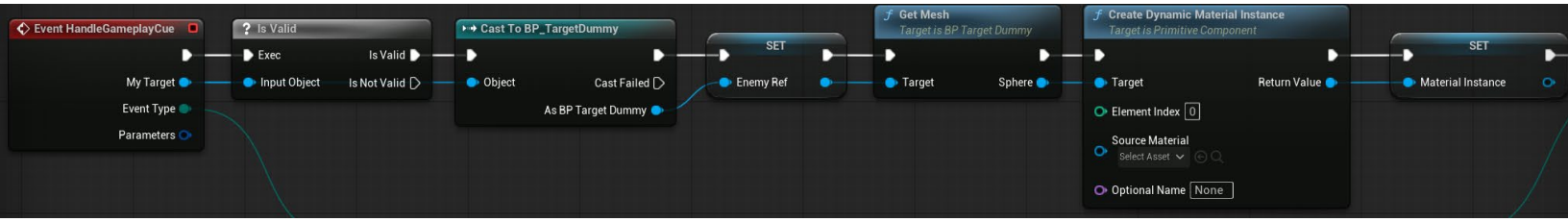


The Cue is a Notify Actor with visual and audio effects:



I use the **HandleGameplayCue** on this one.

After validating the Target, I get the enemy reference, as well as its material instance:



On Active:

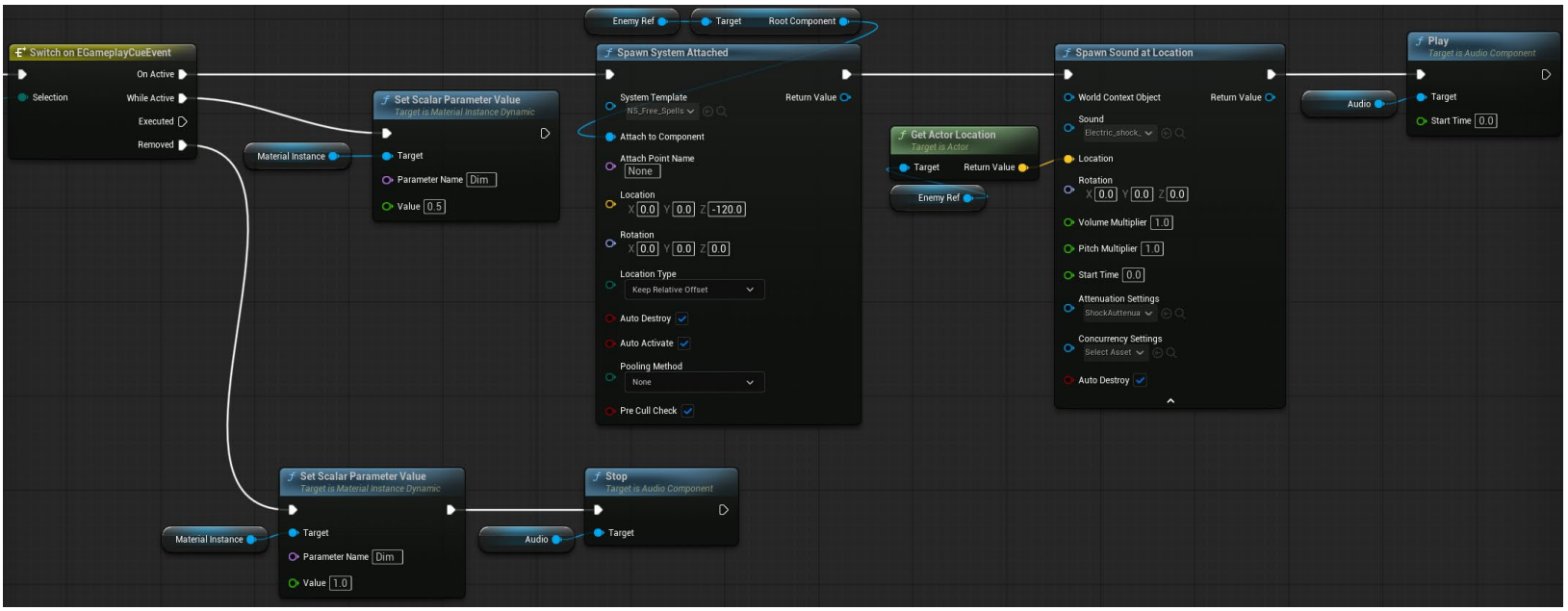
- I spawn an electric effect, like a mini explosion
- A zap sound effect
- Strat playing the electrocution looping sound effect

While Active:

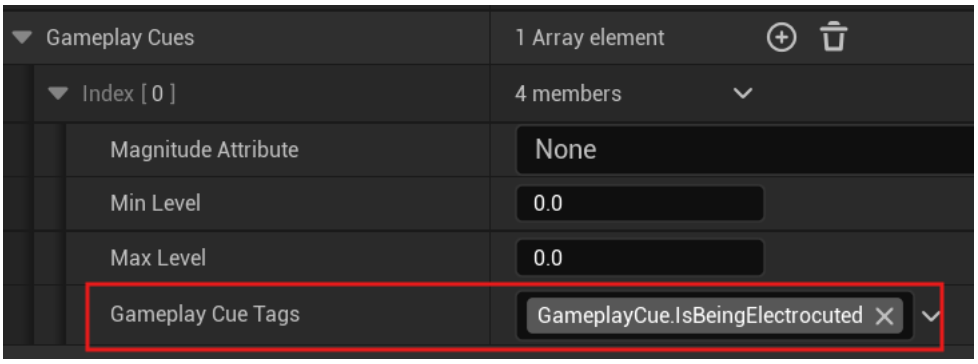
- I set a dimming effect on the material of the enemy

Removed:

- Restore the material's brightness
- Stop playing the looping electrocution sound effect



The GC is associated with a corresponding Gameplay Effect GE_Electrocuted, so it removes itself when the effect is finished:



The Explosion

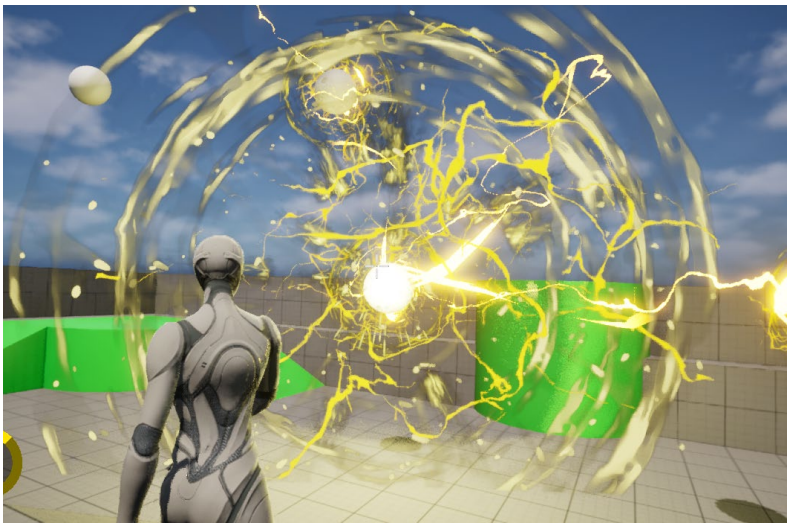
When the lightning projectile collides with something, it generates an explosion.
I used the InFamous games as a reference on how a powerful lightning explosion would look like.

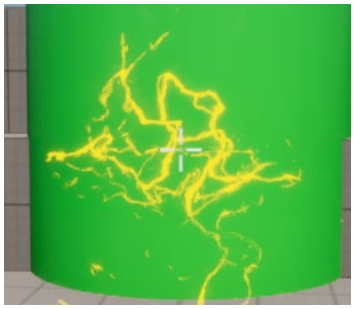
In order for it to feel powerful, I focused on:

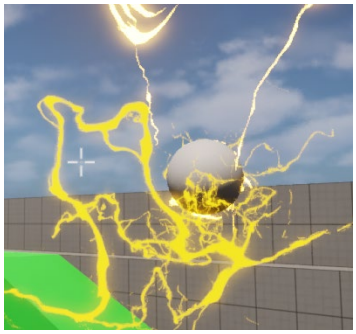
- It must feel explosive
- It must be big
- It must have an effect in the environment

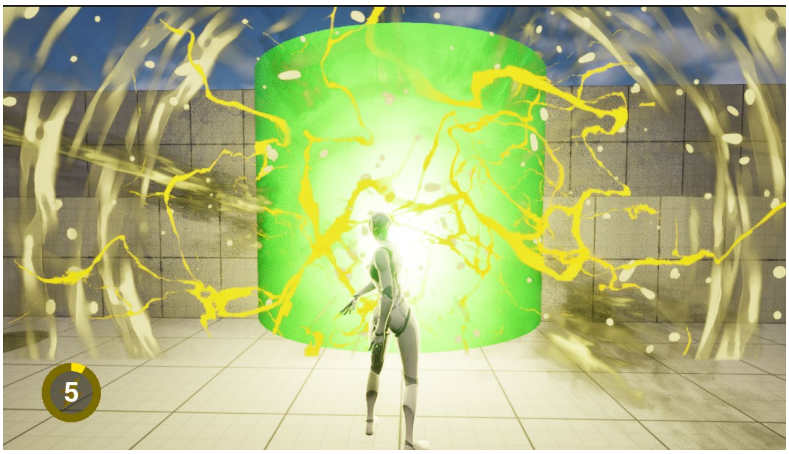
I believe this is achieved by:

- The explosion needs to be heard from far away
- The epicenter must produce a lot of light
    - But I wasn't able to make a flashbang visual effect, so I just used a Point Light
- It leaves a lingering effect after it is gone
- The camera should shake the closer you are to it







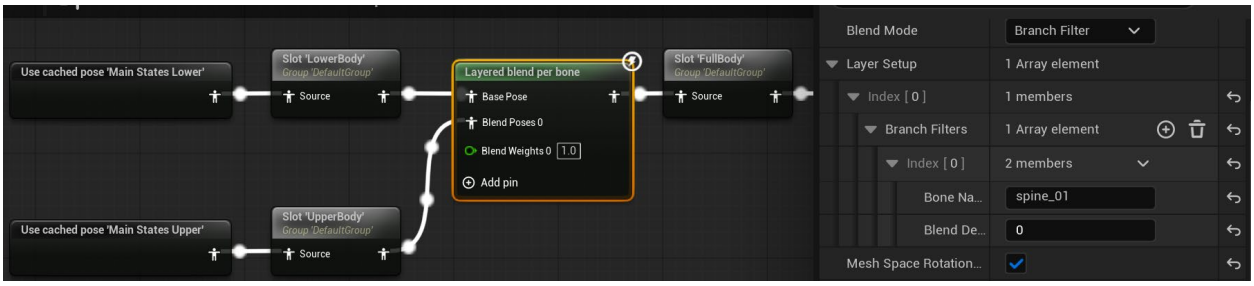


Animation

One of my passions in Game Development is working on the Character. That's why for this project, I also worked on making sure the animations associated with the ability worked like in a real scenario.

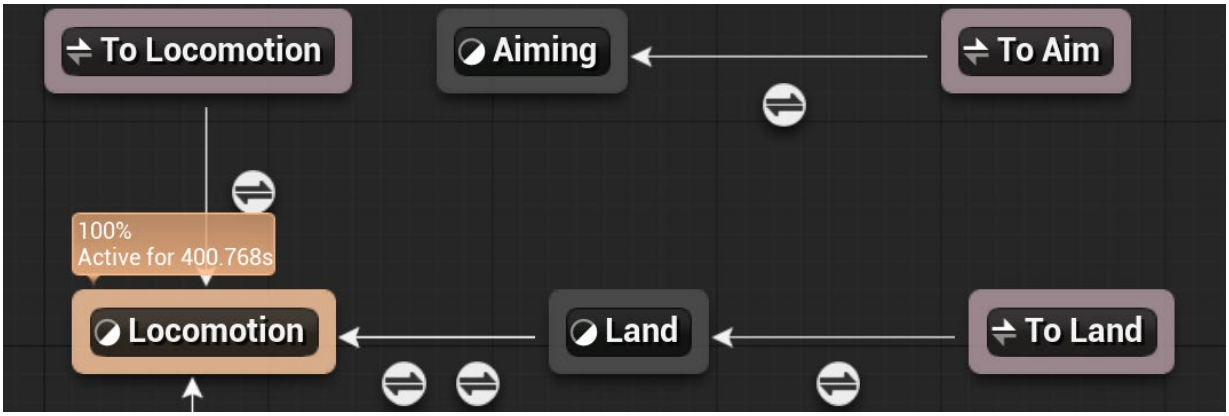
Upper and Lower Body

Because aiming needs to work regardless of your movement, I separated the upper and lower body animations, so when the player aims, the legs keep moving normally, while the torso and up stay in the aiming position.

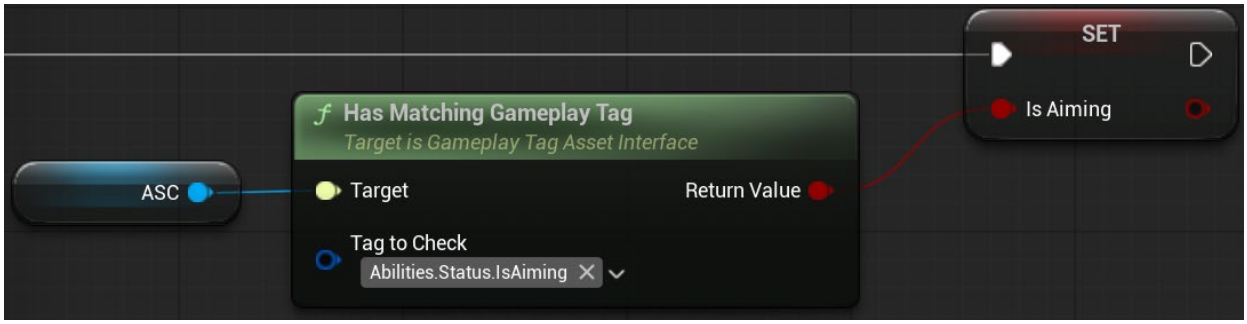


I chose spine_01 as the bone to separate animations on.

The Lower State has the default animations, while the Upper State has the Aim animation included:



And take the Aiming state right from the Ability System Component:



Bone Modification

Since the character is able to aim up and down, I made a replicated variable that returns the pitch angle that you are looking at:

```
UPROPERTY(ReplicatedUsing = OnRep_Pitch)
float PitchInput = 0.0f;
```

It works essentially the same as with the bIsAiming variable.
It gets modified during the DoLook function:

```
void AChainLightningPowerCharacter::DoLook(float Yaw, float Pitch)
{
    if (GetController() != nullptr)
    {
        AddControllerYawInput(Yaw);
        AddControllerPitchInput(Pitch);

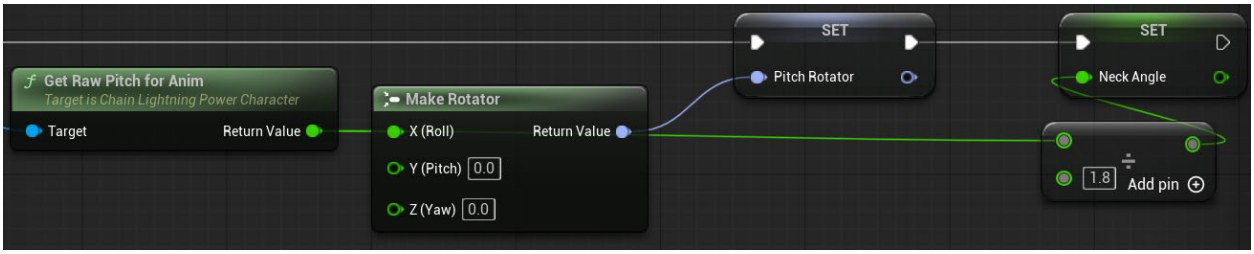
        const float CurrentPitch = FRotator::NormalizeAxis(GetControlRotation().Pitch);
        if (!FMath::IsNearlyEqual(A: CurrentPitch, B: PitchInput, ErrorTolerance: 0.5f))
        {
            PitchInput = CurrentPitch;
            if (!HasAuthority())
            {
                ServerSetPitch(CurrentPitch);
            }
        }
    }
}
```

I use the **NormalizeAxis** to get a value between -180 and 180, as well as the **IsNearlyEqual** to not process too small changes.

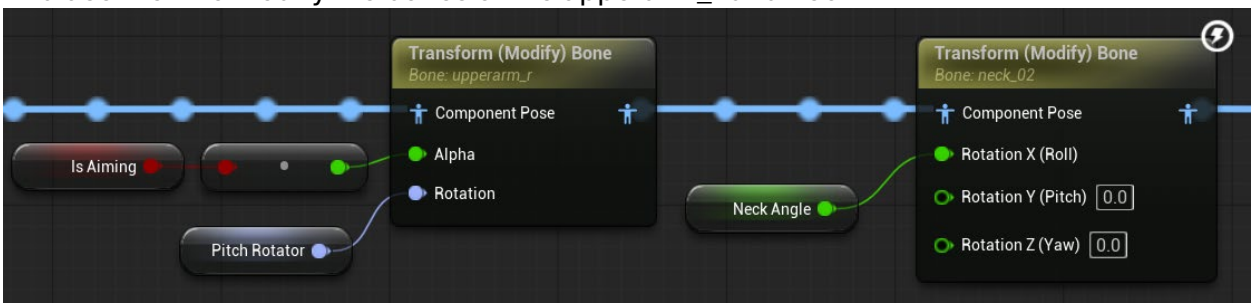
And finally, a function to be called inside the AnimGraph that will return the needed values:

```
float AChainLightningPowerCharacter::GetRawPitchForAnim() const
{
    if (IsLocallyControlled())
    {
        return -FRotator::NormalizeAxis(GetControlRotation().Pitch);
    }
    return -PitchInput;
}
```

I save this variable inside a Rotator and also only the X value isolated, which I divide by 1.8 so it doesn't move the neck too much:



And use them to Modify the bones of the upperarm_r and neck



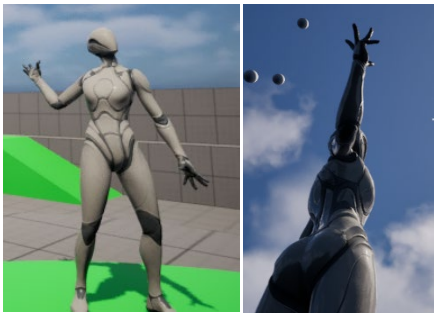
The arm is only modified while aiming.

These make It possible to always move the character's neck to face where you are looking, and aim the arm towards the target point, ensuring aiming consistency.

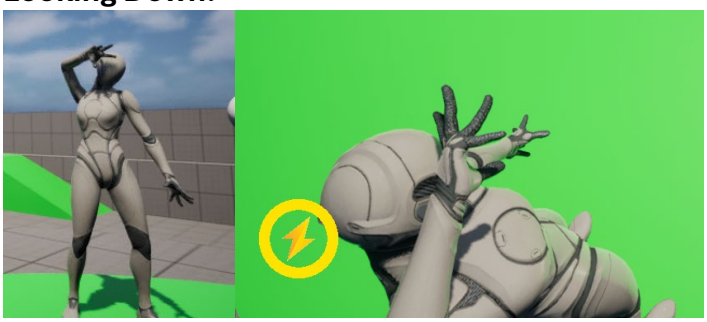
Looking Straight:



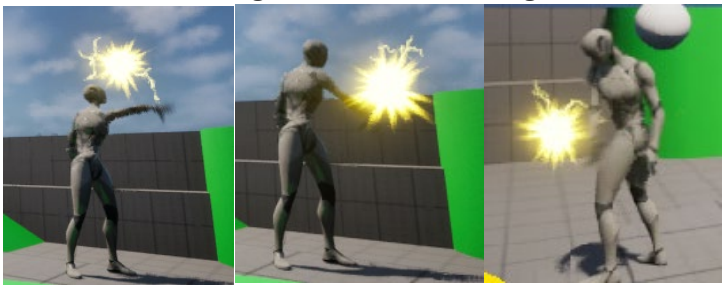
Looking Up:



Looking Down:



The arm will be angled towards the target location:



And for a better aiming angle, I limited the pitch to a 70° and 60° angle up and down respectively:

```
void AChainLightningPowerCharacter::NotifyControllerChanged()
{
    Super::NotifyControllerChanged();

    if (APlayerController* PC = Cast<APlayerController>(Controller); PC && PC->PlayerCameraManager)
    {
        PC->PlayerCameraManager->ViewPitchMin = -60.0f;
        PC->PlayerCameraManager->ViewPitchMax = 70.0f;
    }
}
```

User Interface

Crosshair

As mentioned at the start of the document, the Aim Ability calls the Player Controller's function to add the User Widget that contains the Crosshair.

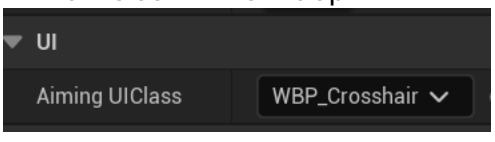
```
if (AChainLightningPowerPlay
{
    PC->ShowAimingUI();
}
```

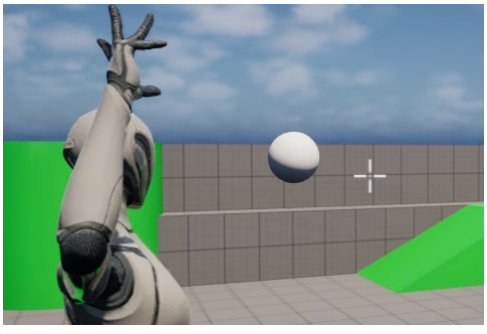
```
void AChainLightningPowerPlayerController::ShowAimingUI()
{
    if (!IsLocalPlayerController() || !AimingUIClass)
    {
        return;
    }

    if (!AimingUI)
    {
        AimingUI = CreateWidget<UAimingUI>( OwningObject: this, AimingUIClass);
    }

    if (AimingUI && !AimingUI->IsInViewport())
    {
        AimingUI->AddToViewport();
    }
}
```

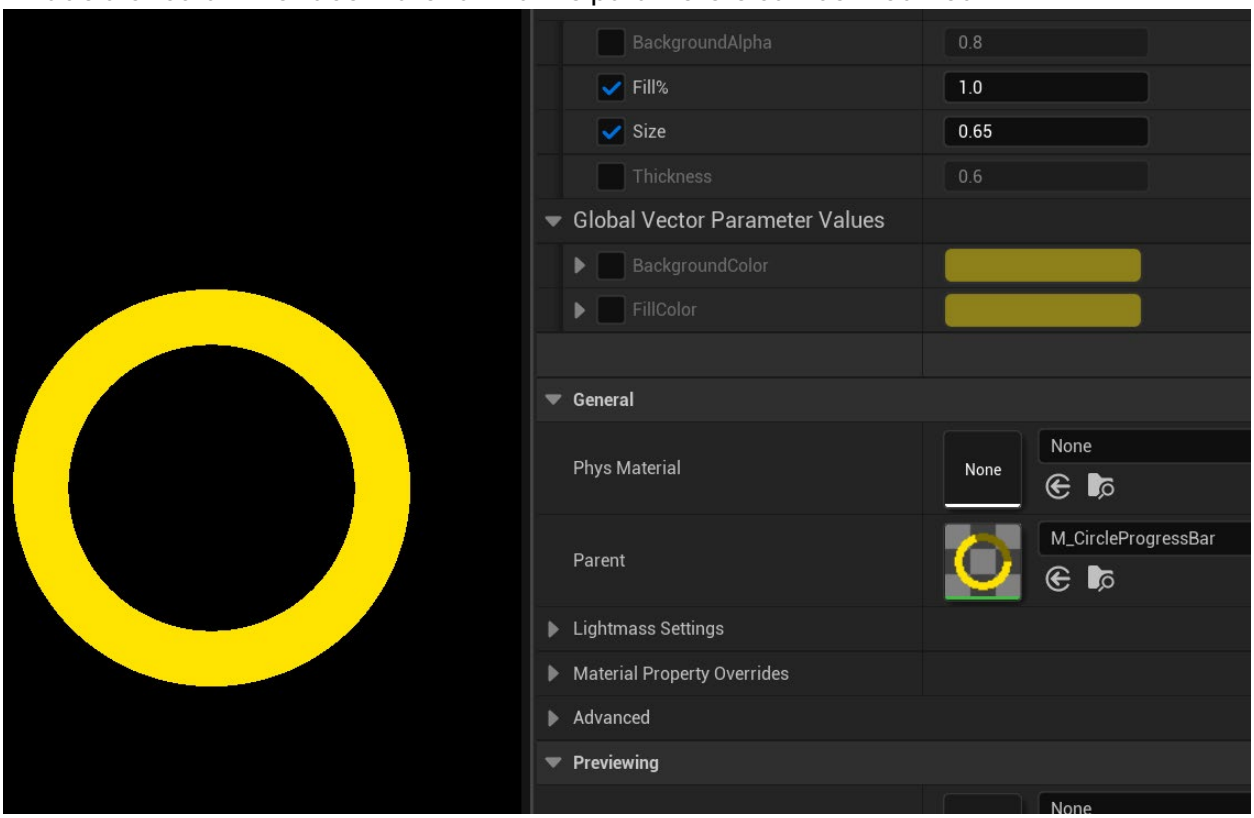
Which is set in the Blueprint:





Cooldown Timer

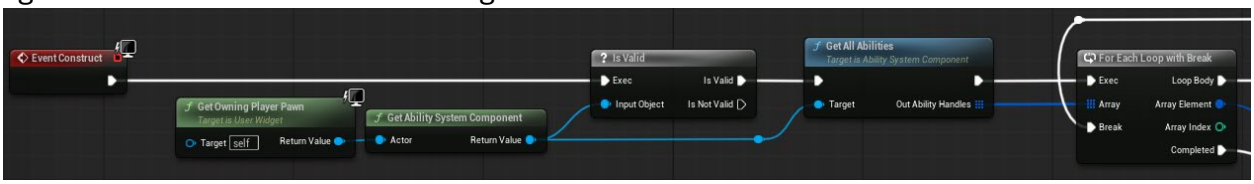
I made a circular interface material that its parameters can be modified:



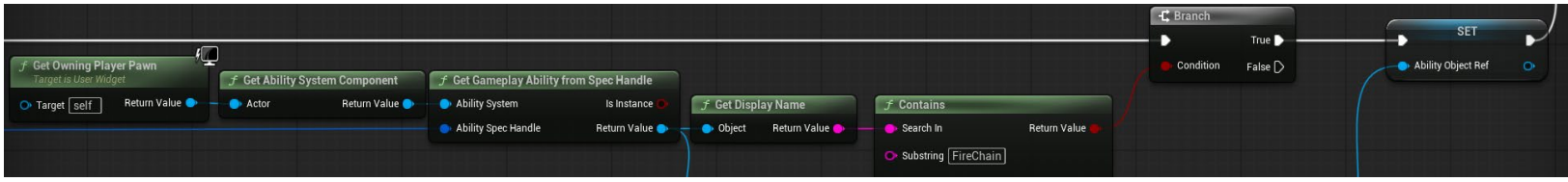
I made a WBP_Cooldown:



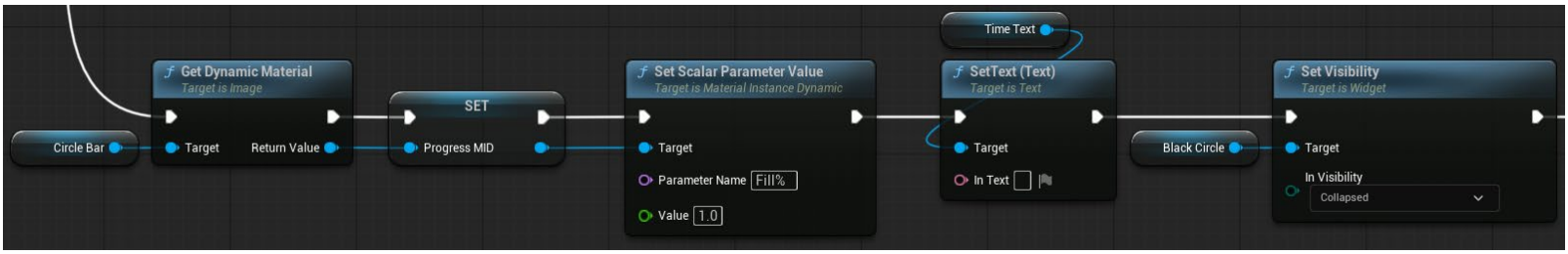
I get all the abilities from the owning Pawn:



And find the one that has “FireChain” to get its Ability Object Reference:



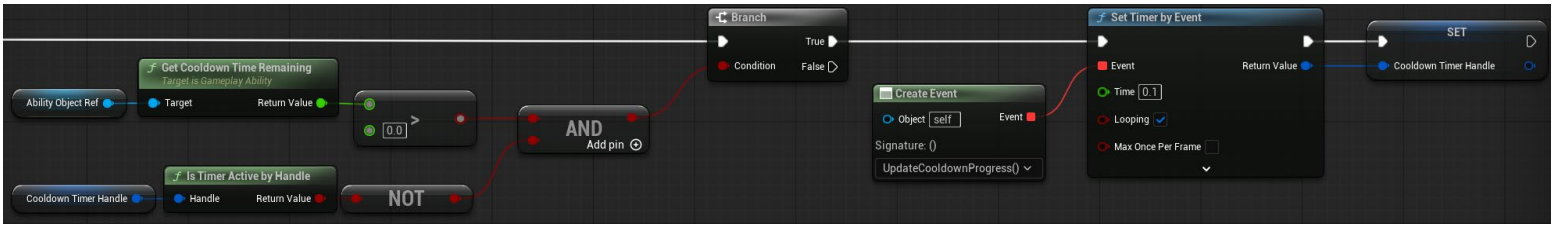
Once it is found, I save its Material Instance to be able to modify the parameters, and set the initial values:



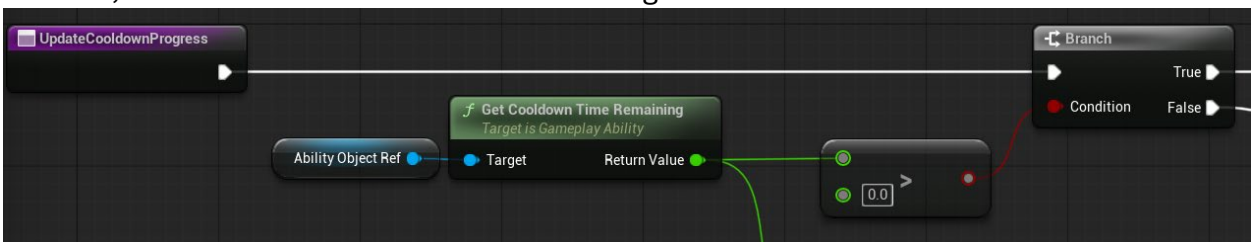
Afterwards, I use Wait Gameplay Count Changed on Actor, using the cooldown Tag:



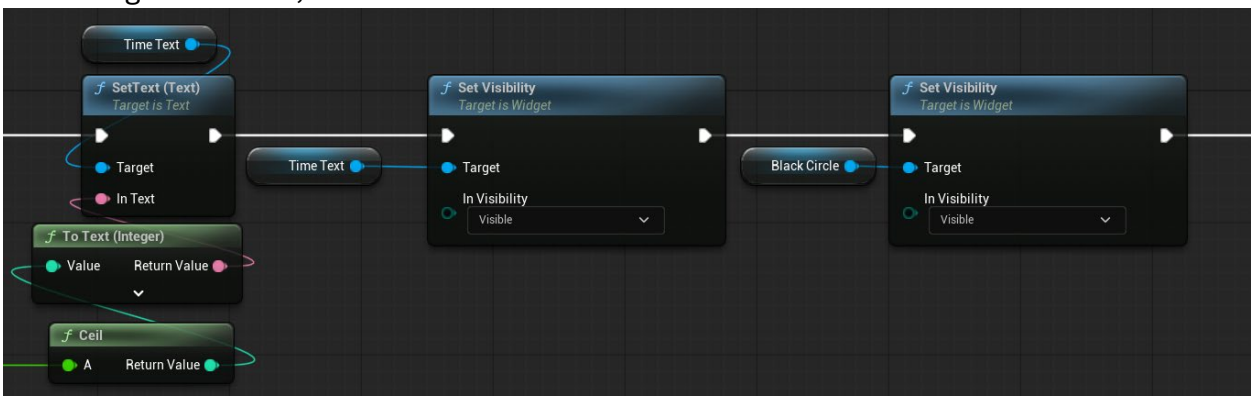
If the Cooldown still has time remaining, and the event timer hasn’t fired yet, I start it by calling the **UpdateCooldownProgress** function:



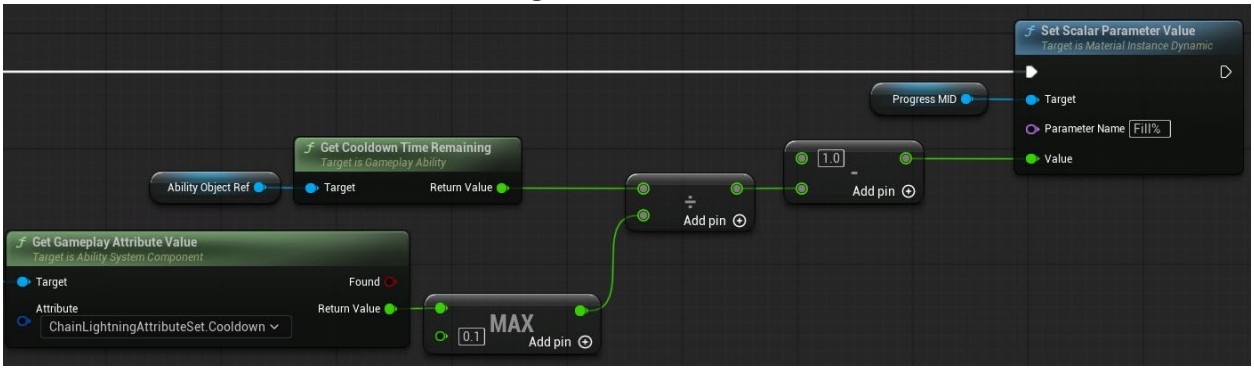
Inside it, I validate the cooldown time remaining:



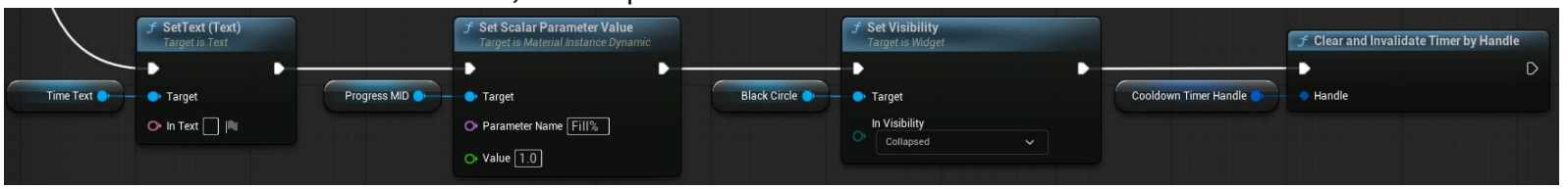
If there is still time remaining, every time this function gets called, I display the time remaining in seconds, and other visual stuff:



And use this time remaining to divide it by the Cooldown time that comes from the attribute set, and subtract 1 - Result to get a value between 0 and 1, where it will be the circle bar not filled at the start of the cooldown, and grow to be filled at the end of it:



Once the timer reaches 0, clean up the timer and return the variables to their default state:



Result:

